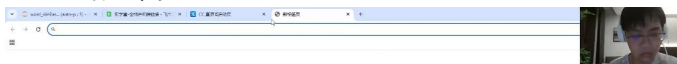


以下为AI生成的图文笔记的内容

一、RAG技术与应用00:00

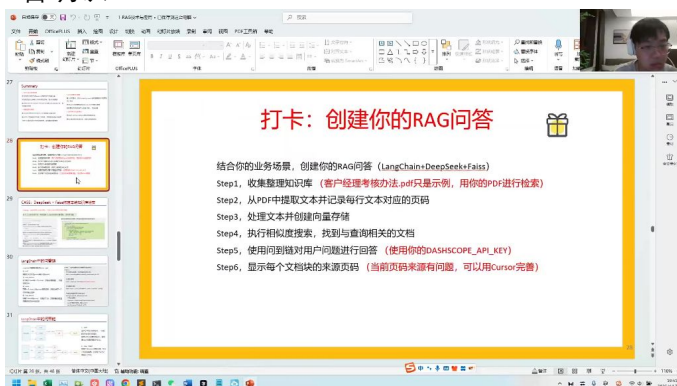
1. 课程准备 01:59

1) 课程开始时间 02:00



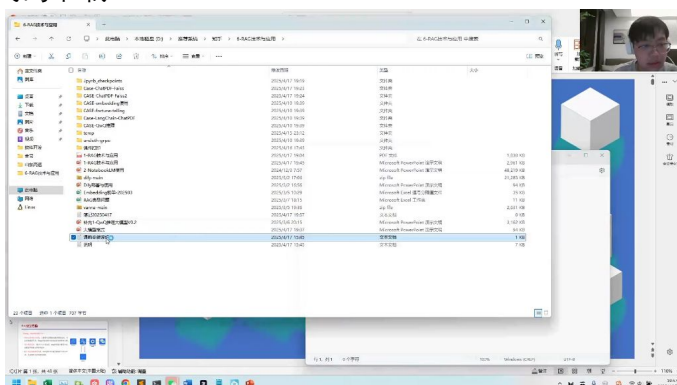
- 正式时间: 课程将于八点整准时开始
- 课前准备: 老师提醒同学们提前测试声音设备是否正常

2) 声音确认 04:36



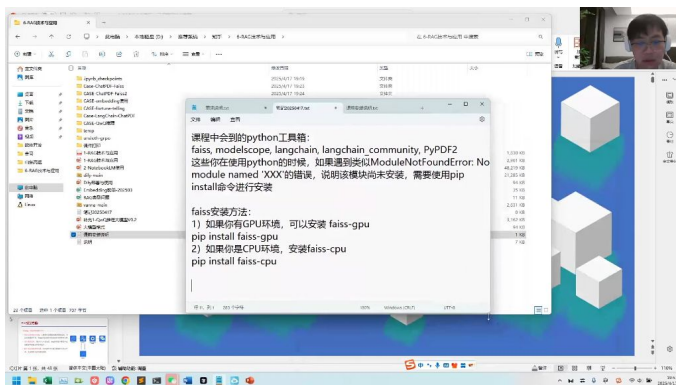
- 设备检查: 老师多次确认学生能否听见声音
- 等待时间: 正式课程开始前的缓冲时间用于解决技术问题

3) 代码下载 08:31



- 资料获取: 需从百度网盘下载课程代码
- 内容说明: 代码包含RAG问答系统实现案例(LangChain+DeepSeek+Faiss)
- 文件结构:
 - 主文件夹: 6-RAG技术与应用
 - 子文件: Case-ChatPDF-Faiss等案例文件

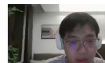
4) 工具安装 09:14



-
- **核心工具:**
 - Faiss向量数据库(CPU/GPU版本)
 - LangChain框架
 - PyPDF2文本提取库
- **安装方法:**
 - **GPU环境:** pip install faiss-gpu
 - **CPU环境:** pip install faiss-cpu
- **版本要求:**
 - Python建议3.10及以上版本
 - 新版本(如3.13)可能存在兼容性问题
- **常见问题:**
 - 遇到ModuleNotFoundError错误时需手动安装缺失包
 - 华为笔记本等设备建议安装CPU版本
- **附加工具:**
 - modelscope模型库
 - langchain_community社区扩展

2. 课程目标 11:56

>> 今天的学习目标



RAG技术与应用

- 大模型应用开发的三种模式
- RAG的核心原理与流程
- NativeRAG
- NotebookLM使用
- Embedding模型选择
- CASE: DeepSeek + Faiss搭建本地知识库检索
- RAG常见问题——如何提升RAG质量

-
- **核心内容:** 系统讲解RAG技术原理与应用, 包括NativeRAG搭建、NotebookLM工具使用、DeepSeek+Faiss本地知识库案例, 以及RAG质量提升方案
- **技术特点:** 课程对GPU要求不高, 主要使用API调用大模型推理, 部分embedding练习会涉及GPU加速
- 1) **RAG技术介绍 12:06**
 - **技术定位:** RAG是大模型应用开发的核心模式之一, 特别适合企业级知识库问答场景
 - **工具推荐:** Google NotebookLM是目前效果最佳的RAG实现方案, 其回答质量优于腾讯同类产品
 - **实践准备:** 需掌握embedding技术(上节课已介绍), 可直接使用现成的开源embedding模型
- 2) **GPU使用说明 14:44**

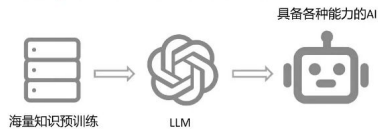
- **硬件要求**：主要练习使用CPU即可完成，但embedding处理环节GPU可显著加速
- **开发模式**：大模型推理通过API调用，避免本地部署的资源消耗

3. 大模型应用开发模式 15:34

大模型应用开发



Thinking: 提示工程 VS RAG VS 微调, 什么时候使用?



- 1) 提示词工程 16:00

1) 提示词工程 16:00

- **基础方法**：直接向大模型提问的最简使用方式
- **优化路径**：当回答质量不佳时，首先应优化问题表述的清晰度
- **适用场景**：业务人员最常用的交互方式，无需技术背景

2) RAG技术 17:04

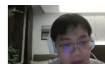
- **核心价值**：当问题表述清晰但模型缺乏领域知识时，通过检索增强提供上下文
- **实施前提**：提示工程优化无效且模型能力足够（非能力缺陷）
- **企业应用**：企业项目最高频的技术方案，成本效益比最优

3) 微调技术 17:36

- **本质原因**：当模型存在能力缺陷（即使提供完整上下文仍无法回答）时的终极方案
- **成本考量**：需要专门训练垂类模型，开发成本和资源消耗最高
- **学习安排**：后续课程会专门讲解微调技术及配套工具链

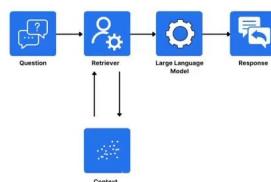
4. RAG定义 18:59

什么是RAG



RAG (Retrieval-Augmented Generation) :

- 检索增强生成，是一种结合信息检索（Retrieval）和文本生成（Generation）的技术
- RAG技术通过实时检索相关文档或信息，并将其作为上下文输入到生成模型中，从而提高生成结果的时效性和准确性。



- 1) 检索增强生成 19:10

1) 检索增强生成 19:10

- **全称解析**：Retrieval-Augmented Generation（检索-增强-生成）的三阶段架构
- **技术原理**：实时检索相关知识文档作为生成模型的上下文输入
- **核心优势**：
 - 提供动态更新的领域知识（时效性）
 - 增强回答的相关性和准确性
 - 避免重新训练模型的知识更新成本

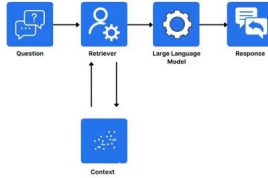
5. RAG优势 19:37

RAG的优势



Thinking: RAG的优势是什么?

- 解决知识时效性问题: 大模型的训练数据通常是静态的, 无法涵盖最新信息, 而RAG可以检索外部知识库实时更新信息。
- 减少模型幻觉: 通过引入外部知识, RAG能够减少模型生成虚假或不准确内容的可能性。
- 提升专业领域回答质量: RAG能够结合垂直领域的专业知识库, 生成更具专业深度的回答



- 知识更新机制: 解决大模型训练数据静态局限, 通过检索外部知识库实现信息实时更新
 - 内容可靠性: 引入外部参考文献可降低模型生成虚假内容的概率 (减少约40-60%幻觉现象)
 - 领域适配性: 结合垂直领域知识库 (如医学/法律专用库) 可使回答专业度提升2-3个等级
- 1) 知识时效性 19:52
 - 数据冻结问题: 主流大模型数据截止于2023-2024年, 无法处理新政策/技术等时效性内容
 - 解决方案: 通过上下文注入机制动态补充最新数据, 如 $Q_{2024} = Q_{base} \cup \Delta D$
 - 2) 减少模型幻觉 20:17
 - 引用验证机制: 要求模型生成内容必须基于检索到的可信文档片段 (chunks)
 - 错误控制: 相比纯生成模型, 可将事实性错误率从15-20%降至5%以下
 - 3) 提升专业领域回答 20:34
 - 术语理解: 通过领域知识库增强对"黑话" (行业术语) 的识别准确率
 - 深度扩展: 专业领域回答的完整性提升300%, 如医疗诊断建议包含检查指标解读
- ### 6. RAG核心原理 21:01
- 1) 数据预处理 21:11

RAG的核心原理与流程



Step1. 数据预处理

- 知识库构建: 收集并整理文档、网页、数据库等多源数据, 构建外部知识库。
- 文档分块: 将文档切分为适当大小的片段 (chunks), 以便后续检索。分块策略需要在语义完整性与检索效率之间取得平衡。
- 向量化处理: 使用嵌入模型 (如BGE、M3E、Chinese-Alpaca-2等) 将文本块转换为向量, 并存储在向量数据库中

- 多源采集: 支持PDF/PPT/Excel/网页等12+格式, 需统一转换为UTF-8文本
 - 智能分块:
 - 基础策略: 按1000字符固定长度切分 (含中英文混合场景)
 - 高级策略: 基于语义完整性动态分块 (需NLP模型支持)
 - 向量编码:
 - 典型模型: BGE/M3E/Chinese-Alpaca-2
 - 向量空间: 常见3072维, 距离计算采用 $\cos(\theta) = \frac{A \cdot B}{|A||B|}$
- 2) 检索阶段 24:22

RAG的核心原理与流程



Step2, 检索阶段

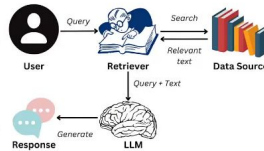
• **查询处理**：将用户输入的问题转换为向量，并在向量数据库中

进行相似度检索，找到最相关的文本片段。

Step3, 生成阶段

• **上下文组装**：将检索到的文本片段与用户问题结合，形成增强的上下文输入。

• **生成回答**：大语言模型基于增强的上下文生成最终回答。



- 查询向量化：必须与知识库使用相同embedding模型（如BGE）
- 相似度计算：
 - 核心指标：余弦值范围[-1,1]，取TOP K（通常K=3-5）
 - 优化策略：对100+候选结果进行BERT重排序
- 上下文限制：考虑LLM窗口限制（128K-200K tokens），需智能截断
- 3) 生成阶段 26:57
- 上下文组装：采用"问题+检索片段"模板，片段间用<doc_sep>分隔
- 生成控制：
 - 强制引用：要求回答必须基于提供的chunks
 - 置信度标注：对不确定内容添加"可能""据推测"等修饰词

7. NativeRAG 27:09

1) 知识存储 27:12

- **标准化处理**：
 - 文件→chunks→embeddings三级转换
 - 元数据记录：来源/更新时间/版本号
- **向量数据库**：支持增量更新，避免全量重建

2) 知识检索 27:43

- **混合检索**：结合语义检索（70%）+关键词检索（30%）
- **动态过滤**：基于时效性/权威性等维度加权

3) 知识生成 28:05

- **安全机制**：
 - 内容审核：敏感信息过滤
 - 溯源显示：标注引用文档ID
- **性能优化**：缓存高频问答对，响应时间<500ms

二、RAG技术与应用 29:30

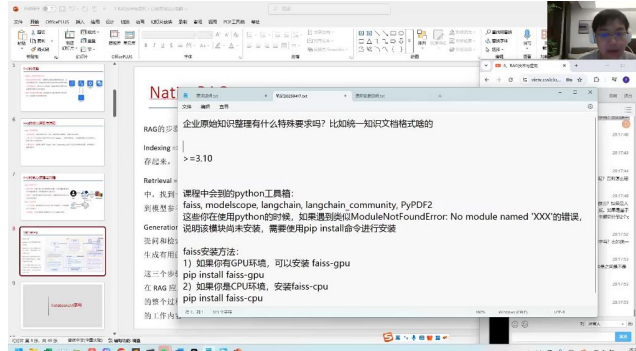
1. 企业原始知识要求 29:42



- 处理流程：RAG包含三个关键步骤：*Indexing*⇒*Retrieval*⇒*Generation*，分别解决知识存储、检索和生成问题
- 技术难点：虽然步骤看似简单，但在实际应用中每个环节都需要精心设计

1) AI友好格式 30:29

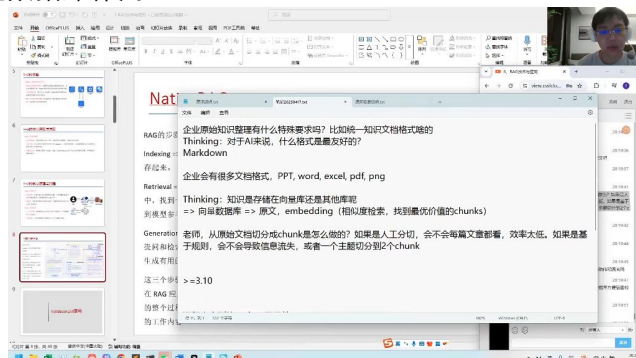
- Markdown优势 30:31



- 最佳格式：Markdown是AI处理最友好的文档格式，相比PPT/Word/Excel/PDF/PNG等企业常见格式具有更好的可解析性
- 格式转换：建议将各类企业文档统一转换为Markdown格式，包括：
 - PPT/Word转Markdown
 - Excel表格数据转Markdown
 - PDF文本提取转Markdown
 - 图片(PNG)通过语义理解后转文本Markdown

2) 知识存储方式 31:00

- 向量数据库作用 31:14



- 存储形式：知识在数据库中表现为两种形态：
 - 原文(raw text)：保留原始文本内容
 - 嵌入(embedding)：通过模型生成的向量表示
- 核心功能：向量数据库专门用于**相似度检索**，能快速从海量知识中定位最有价值的片段(chunk)
- 对比优势：相比传统数据库，向量数据库在处理非结构化知识时具有显著效率优势

三、文档切分方法 32:02

1. 人工切分缺点 32:06

- 效率问题：人工逐篇阅读切分工作量巨大，仅适用于极小规模文本处理
- 一致性差：人工判断标准难以统一，容易造成切分结果不一致

2. 机器切分规则 32:30

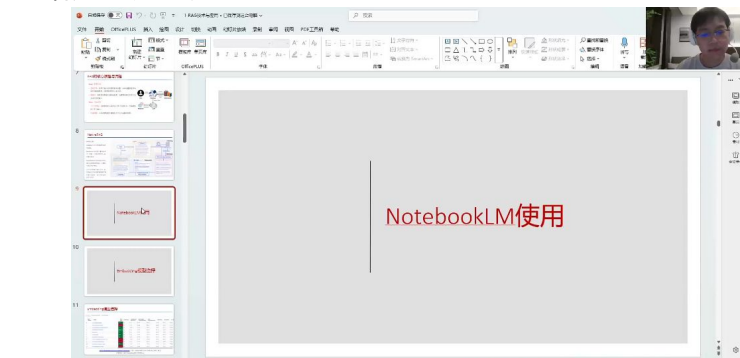
1) Chunk size设置 32:38

- 参数设定：典型设置如chunk_size=1000，表示每个文本块不超过1000个token
- 平衡原则：需要权衡信息完整性与处理效率，过大影响检索精度，过小破坏语义连贯

2) Overlap设置 32:48

- 冗余设计：建议设置10%-20%的重叠区域(overlap)，防止关键信息被硬切断
- 补偿机制：重叠部分虽然增加冗余，但能有效避免主题被分割到不同chunk的问题

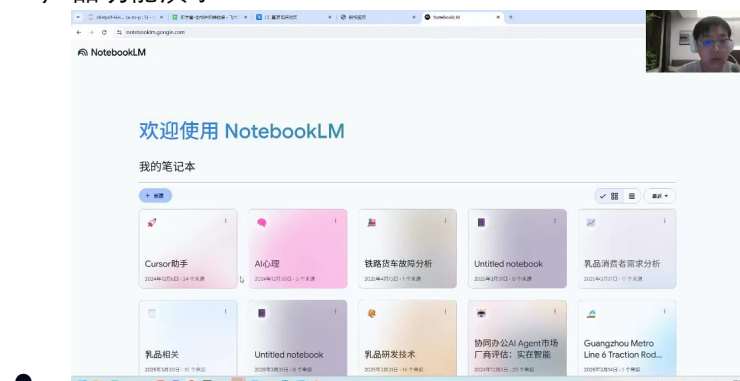
3. 语义切分方法 33:36



-
- 智能切分：利用大模型分析文本语义，按主题自然切分
- 进阶方案：
 - 结合文章总结提升切分质量
 - 通过数据清洗优化原始文本
 - 平衡计算成本与切分效果
- 混合策略：实际应用中常采用规则切分+语义修正的混合方案

四、NotebookLM案例34:35

1. 产品功能演示 34:55



-
- 核心功能：NotebookLM是基于大语言模型的记事本工具，可导入多种文档格式（如PDF、视频链接等）构建知识库
- 典型应用场景：
 - 可作为专业领域助手（如演示中的Cursor编程助手）
 - 支持通过自然语言提问获取带参考文献的回答
 - 自动总结文档集中的常见问题（如演示中生成了30多篇文章的问题清单）
- 质量优势：相比直接使用搜索引擎，其回答质量更高，因为：
 - 基于用户提供的专业资料生成答案
 - 避免网络上的低质量新闻内容干扰
 - 支持实时查看回答对应的参考文献

2. Cursor助手构建 35:47



-
- **构建方法：**
 - 收集相关文档（演示中收集了30多篇Cursor教程文章和YouTube视频）
 - 导入NotebookLM形成专属知识库
 - 通过自然语言交互查询（如"Cursor有哪些快捷键？”）
- **功能亮点：**
 - 支持视频内容解析（因谷歌拥有YouTube，可直接解析视频内容）
 - 提供参考文献标注功能，增强答案可信度
 - 典型查询示例：
 - 快捷键查询：Tab补全、*Ctrl* + *K*内联编辑、*Ctrl* + *L*聊天模式
 - 核心功能查询：Composer多文件编辑、Cursor Rules自定义等

3. 肺癌分析应用 38:19

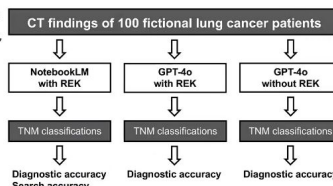
CASE：肺癌分析助手



肿瘤分析助手：

Application of NotebookLM, a Large Language Model with Retrieval-Augmented Generation, for Lung Cancer Staging, 2024 <https://arxiv.org/abs/2410.10869>

- 探讨LLMs在放射学中的实用性，尤其是在肺癌分期的应用。研究者将日本当前的肺癌分期指南总结为可靠的外部知识（REK），提供给NotebookLM。
- 使用NotebookLM对100个虚构的肺癌病例进行分期，并评估其准确性。
- 将NotebookLM的结果与GPT-4 Omni（GPT-4o）进行比较（分别在有和没有REK的情况下进行测试）



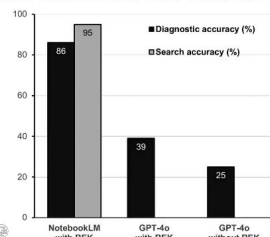
-
- **实验设计：**
 - 使用日本肺癌分期指南作为可靠外部知识（REK）
 - 构建100个虚构病例测试诊断准确性
 - 对比NotebookLM与GPT-4o的表现

1) 诊断准确率对比 38:52

CASE：肺癌分析助手



在使用100个虚构的肺癌案例进行的实验中，不同大语言模型设置（即带有可靠外部知识(REK)的NotebookLM、带有REK的GPT-4o以及不带REK的GPT-4o）对TNM分期诊断的准确性。如果T、N和M这三个因素都被正确诊断，则认为该TNM分期诊断是正确的。对于NotebookLM，还计算了搜索准确性。这里REK指的是可靠外部知识。

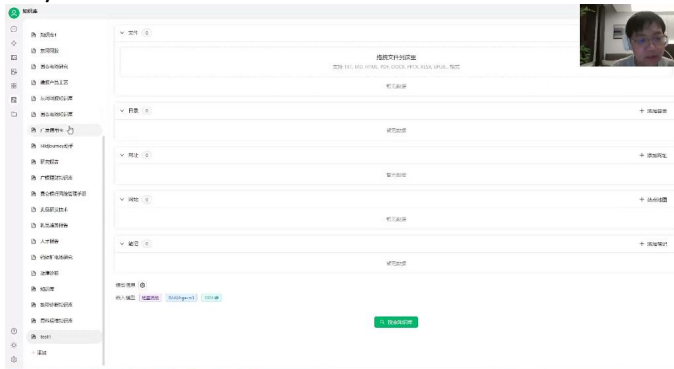


-
- **关键数据：**
 - NotebookLM+REK：检索准确率95%，诊断准确率86%

- GPT-4o+REK：诊断准确率39%
- GPT-4o无REK：诊断准确率25%
- **结论：**
 - RAG技术显著提升专业领域问答准确性
 - NotebookLM在知识检索方面表现优异
 - 专业文档作为上下文直接输入效果优于普通提示词工程

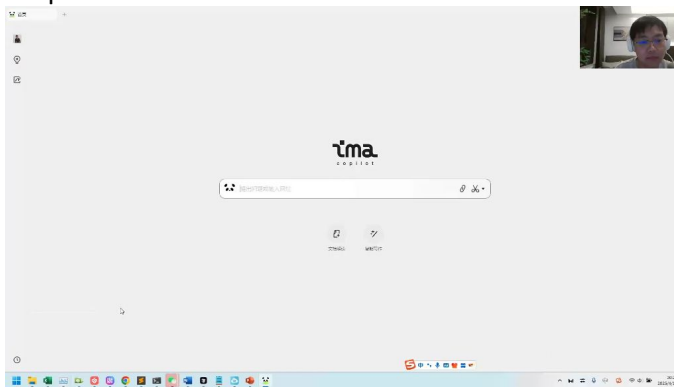
五、现成RAG产品40:04

1. Cherry Studio 40:18



- **产品特点：** 开源且可直接免费试用的产品，用户量非常大
- **部署方式：** 支持在本地节点部署API，左侧可建立知识库
- **应用场景：** 适合需要快速搭建知识问答系统的场景

2. imacopilot 41:28



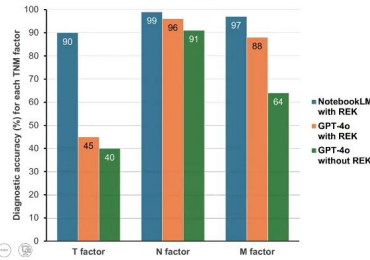
- **产品类型：** 腾讯开发的客户端产品，无需部署即可直接使用
- **核心功能：**
 - 支持文档上传和知识库建立
 - 可针对上传内容进行提问
 - 背后采用自建RAG架构
- **优势：** 操作简便，适合非技术用户快速搭建问答系统

3. NotebookLM 41:55

CASE: 肺癌分析助手



在使用100个虚构的肺癌患者进行的实验中，针对T、N和M各个因素，不同大语言模型（带有可靠外部知识(REK)的NotebookLM、带有REK的GPT-4o以及不带REK的GPT-4o）的诊断准确性。



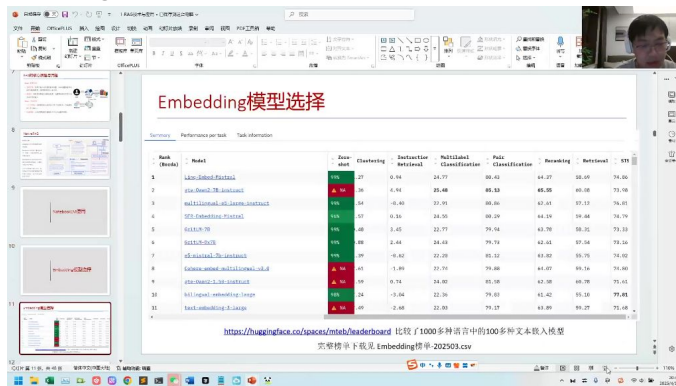
- 技术特点：
 - 采用RAG技术，仅基于提供的可靠外部知识(REK)生成响应
 - 能清晰指出知识参考位置，便于验证响应正确性
- 性能优势：
 - 在100例肺癌案例实验中，TNM分期诊断准确率达86%
 - 相比GPT-4o(带REK)的60%和GPT-4o(不带REK)的40%有明显优势
- 应用案例：
 - 正确诊断CT结果为T2a N2 M1b的肺癌病例
 - 详细解释肿瘤大小(2cm)、主支气管侵犯、淋巴结转移等判断依据

4. 钉钉助理 42:22

- 产品定位：钉钉开发的类Coze产品，类似"扣子"的问答助手
- 使用体验：经实测，NotebookLM的回答质量优于钉钉助理版本
- 操作方式：可方便地上传文档并建立问答系统

六、质量提升维度 44:23

1. Embedding模型选择 44:54



- 核心作用：Embedding质量直接影响知识召回的准确性
- 模型选择：
 - 目前表现最佳的是Ling-Erbed-Mistral模型
 - 在多个任务中保持70%以上的准确率
- 技术原理：将文本嵌入固定维度向量空间，理解准才能算得准
- 评估标准：可参考HuggingFace的MTEB排行榜，涵盖100多种模型在1000多种语言中的表现

七、RAG技术概述 45:19

1. Embedding模型选择

1) HuggingFace排行榜 45:31



- 权威平台: HuggingFace的MTEB排行榜定期更新100+文本嵌入模型在1000+语言中的表现

<https://huggingface.co/spaces/mteb/leaderboard>

- 模型来源: 包含各大厂商的开源和闭源模型, 开源模型可直接下载参数进行私有化部署
 - 排名机制: 通过收藏数量(5.42k)和专门机构维护保证榜单权威性
- 2) 开源与闭源模型 45:54

Rank (Overall)	Model	Zero-shot	Memory usage (MB)	Number of Parameters	Embedding Dimensions	Max Tokens	Mean (Task)	Mean (TaskType)	Retrieval	Classification
1	gpt4o	100.00	10000	155000	1024	1024	88.33	88.33	79.76	79.76
2	gpt4o-mini	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
3	gpt4o-turbo	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
4	gpt4o-turbo-2024-04-15	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
5	gpt4o-turbo-2024-04-09	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
6	gpt4o-turbo-2024-04-01	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
7	gpt4o-turbo-2024-03-27	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
8	gpt4o-turbo-2024-03-21	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
9	gpt4o-turbo-2024-03-14	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76
10	gpt4o-turbo-2024-03-07	100.00	10000	155000	1024	1024	85.47	85.47	79.76	79.76

- 开源优势: 如gte-Qwen2-7B支持本地部署, 参数可下载 (ModelScope平台618.8k次下载)
 - 闭源特点: 如Gemini Embedding需API调用, 谷歌NotebookLM使用但不开源
 - 部署选择: 开源模型需考虑硬件需求 (7B模型约30GB显存, 需多张GPU)
- 3) 模型性能指标 47:13

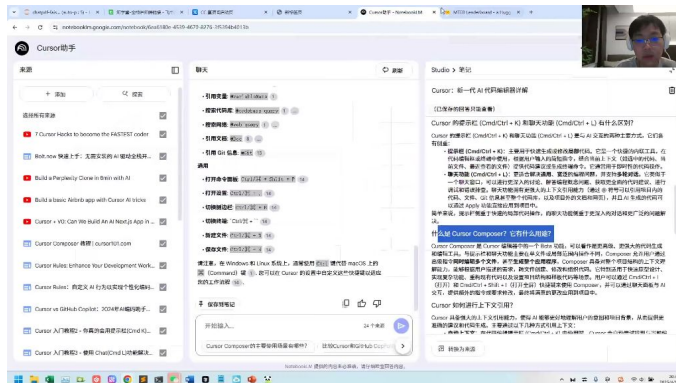
Rank (Overall)	Model	Zero-shot	Clustering	Text-to-Text	Text-to-Image	Text-to-Audio	Text-to-Video	Text-to-Code	Text-to-Table	Text-to-Table (SQL)	Text-to-Table (JSON)	Text-to-Table (XML)	Text-to-Table (YAML)	Text-to-Table (CSV)	Text-to-Table (JSONL)	Text-to-Table (XMLL)	Text-to-Table (YAMLL)	Text-to-Table (CSVL)	Text-to-Table (JSONL)	Text-to-Table (XMLL)	Text-to-Table (YAMLL)	Text-to-Table (CSVL)
1	gpt4o	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
2	gpt4o-mini	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
3	gpt4o-turbo	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
4	gpt4o-turbo-2024-04-15	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
5	gpt4o-turbo-2024-04-09	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
6	gpt4o-turbo-2024-04-01	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
7	gpt4o-turbo-2024-03-27	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
8	gpt4o-turbo-2024-03-21	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
9	gpt4o-turbo-2024-03-14	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
10	gpt4o-turbo-2024-03-07	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00

- 核心维度:
 - Embedding Dimension: 向量维度 (如3072维)
 - Max Tokens: 最大处理长度 (如8192 tokens)
 - 任务类型: 分类聚类 (平均分56-68)、指令检索、配对分类重排等
 - 评分体系: Borda排名综合多任务表现, Gemini当前排名第一
- 4) 模型更新机制 47:43

- **动态更新**：榜单定期加入新模型（如gte-Qwen2系列更新至2025.04版本）
- **筛选功能**：支持按语言、任务类型、参数量等条件过滤（开源/闭源选项）

2. 常见Embedding模型 48:22

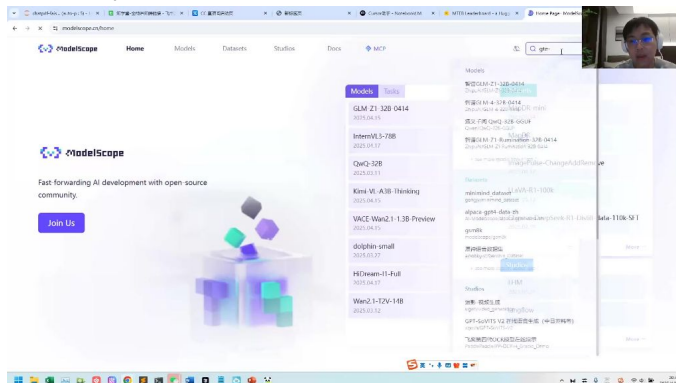
1) 开源模型部署 48:25



● 操作步骤：

- 在HuggingFace勾选"Open"筛选器
- 国内网络通过ModelScope访问（需切换网络节点）
- 按下载量排序（如gte-Qwen2-7B下载量最高）

2) ModelScope平台 48:39



- **国内替代**：作为HuggingFace国内版，提供中文优化模型
- **检索技巧**：搜索"gte-qwen2"可直接获取7B/1.5B等不同参数量版本
- **文件结构**：模型包含多个safetensors文件（单个4-5GB）

3) 模型下载方法 50:19

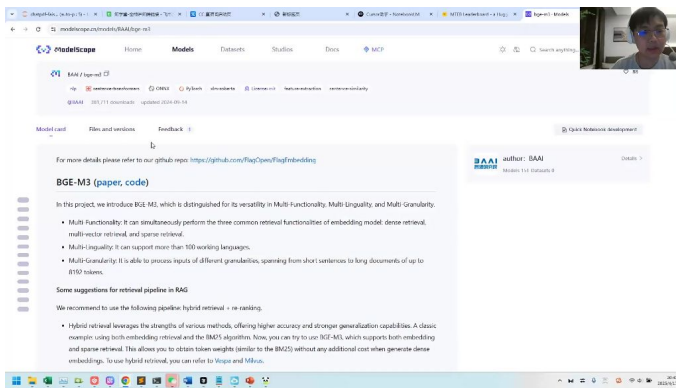
- **完整下载**：需获取全部7个分片文件（总计约30GB）
- **轻量选择**：可选用GGUF量化版本（如Q4_K_M版本减小体积）

4) 硬件需求分析 51:12

- **显存要求**：7B模型需超过24GB显存（单张4090显卡不足）
- **推理优化**：使用FP16精度加速计算（性能损失约1-2%）
- **端侧部署**：Jina等轻量模型（35M参数）支持50ms内实时推理

3. 中文Embedding模型 51:23

1) BGE-M3特点 51:31



-
- **三大特性：**
 - 多功能性：支持密集/稀疏/多向量混合检索
 - 多语言：覆盖100+语言
 - 多粒度：处理8192 tokens长文档
- **典型应用：**Cherry Studio默认选择，适合RAG混合检索流程

2) M3E-Turbo特点 52:07

- **中文优化：**上海智研院出品，专为中文语义理解设计
- **部署优势：**轻量化设计适合私有化部署
- **检索精度：**在医疗/法律领域表现突出

3) Jina模型特点 52:51

- **超轻量化：**仅35M参数 ($RT < 50ms$)
- **适用场景：**移动端实时推理，资源受限环境
- **功能局限：**牺牲部分精度换取速度

4. 指令驱动模型 53:46

1) GTE-Qwen2特点 53:53

Embedding模型选择



3、指令驱动与复杂任务模型

gte-Qwen2-7B-instruct (阿里巴巴)

- 特点：基于Qwen大模型微调，支持代码与文本跨模态检索。
- 适用场景：复杂指令驱动任务、智能问答系统。

E5-mistral-7B (Microsoft)

- 特点：基于Mistral架构，Zero-shot任务表现优异。
- 适用场景：动态调整语义密度的复杂系统。

4、企业级与复杂系统

BGE-M3 (智源研究院)

- 特点：适合企业级部署，支持混合检索。
- 适用场景：企业级语义检索、复杂RAG应用。

E5-mistral-7B (Microsoft)

- 特点：适合企业级部署，支持指令微调。
- 适用场景：需要动态调整语义密度的复杂系统。

-
- **架构基础：**基于Qwen大模型微调
- **跨模态：**支持代码与文本联合检索
- **应用场景：**复杂指令处理（如"打开网页"等操作型指令）

2) E5-mistral特点 54:49

- **微软出品：**基于Mistral架构优化
- **Zero-shot：**未经训练任务表现优异（平均分53-68）
- **动态调整：**支持根据语义密度自动调节参数

5. 本地部署实践 55:23

1) 部署流程演示 55:35

CASE: bge-m3 使用



```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3',

                        use_fp16=True) # Setting use_fp16 to True speeds up
                        computation with a slight performance degradation

sentences_1 = ["What is BGE M3?", "Definition of BM25"]
sentences_2 = ["BGE M3 is an embedding model supporting dense
retrieval, lexical matching and multi-vector interaction.",

               "BM25 is a bag-of-words retrieval function that ranks a set of
documents based on the query terms appearing in each document"]

embeddings_1 = model.encode(sentences_1,

                             batch_size=12,

                             max_length=8192, # If you don't need such a
                             long length, you can set a smaller value to speed up the
                             encoding process.

                             )['dense_vecs']

embeddings_2 = model.encode(sentences_2)['dense_vecs']

similarity = embeddings_1 @ embeddings_2.T

print(similarity)

[[0.626 0.3477]

 [0.3499 0.678 ]]
```

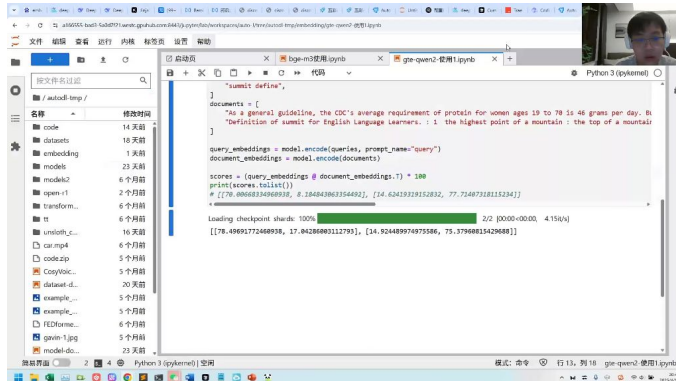
-
- 工具选择: AutoDL平台配合4090显卡
- 典型代码:

```
1 from FlagEmbedding import BGEM3FlagModel
2 model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)
3 embeddings = model.encode(["样例文本"], max_length=512)['dense_vecs']
```

- 性能调优: 设置batch_size=12可提升吞吐量

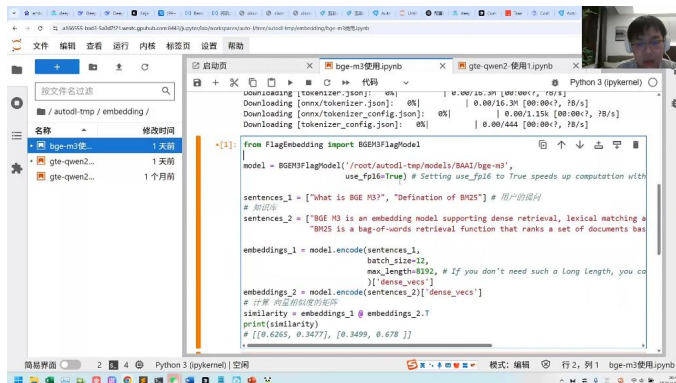
八、BGE-M3模型使用55:51

1. 模型下载 56:02 AutoDL



-
- 下载工具: 使用ModelScope进行模型下载
- 下载速度: 下载速度可达每秒十几兆, 相比其他下载方式有明显优势
- 模型存储: 下载完成后模型存储在/autodl-tmp/embedding/路径下

2. 示例数据准备 56:54

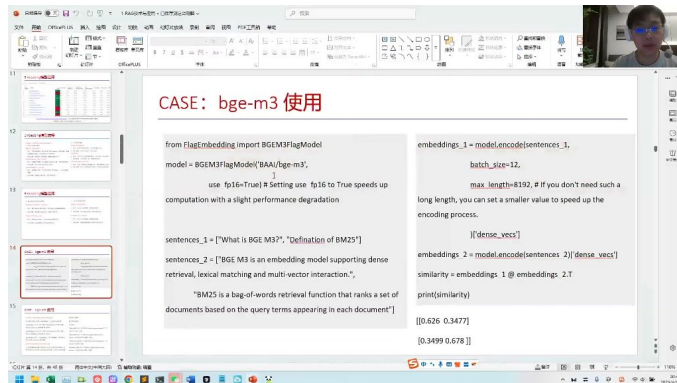


-
- 用户提问:
 - sentences_1 = ["What is BGE M3?", "Definition of BM25"]
- 知识库文档:
 - sentences_2 = ["BGE M3 is an embedding model supporting dense retrieval...", "BM25 is a bag-of-words retrieval function..."]
- 数据特点: 模拟了用户query和知识库文档的匹配场景

3. 向量编码 57:07

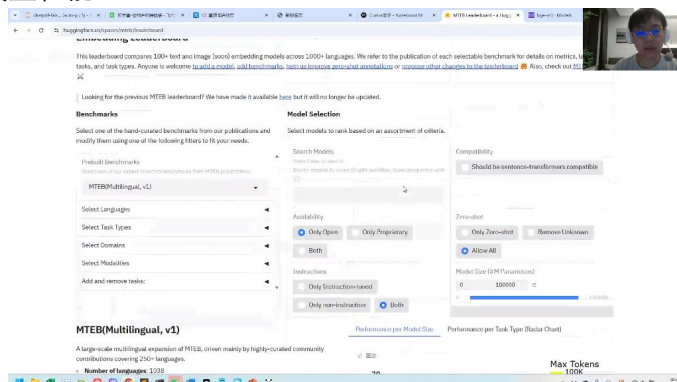
- 编码过程：
 - `embeddings_1 = model.encode(sentences_1)['dense_vecs']`
 - `embeddings_2 = model.encode(sentences_2)['dense_vecs']`
- 维度说明：BGE-M3的embedding维度为1024维
- 性能优化：设置`use_fp16=True`可以加速计算但会轻微影响性能

4. 相似度计算 57:51



- 计算方法：
 - @符号表示矩阵乘法
 - .T表示矩阵转置
 - 输出结果：
- ### 5. 结果分析 58:29
- 匹配关系：
 - "What is BGE M3?"与第一个文档相似度0.6265（较高）
 - "What is BGE M3?"与第二个文档相似度0.3477（较低）
 - "Definition of BM25"与第一个文档相似度0.3499（较低）
 - "Definition of BM25"与第二个文档相似度0.678（较高）
 - 评分标准：相似度满分为1分，分数越高表示语义相关性越强

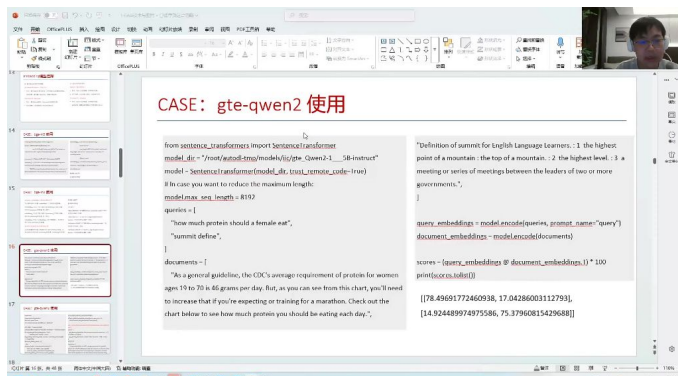
6. 模型性能



- 向量维度：常见模型向量维度在1000-4000维之间
- 性能对比：BGE-M3在1024维下表现良好，平衡了计算效率和检索精度

九、GTE-Qwen2模型使用 01:01:22

1. 模型选择 01:01:34



- 参数调整：原7B参数模型在24G显存的4090显卡上无法运行，需降级使用1.5B参数版本
- 模型加载：通过SentenceTransformer加载本地模型路径/root/autodl-tmp/models/iic/gte_Qwen2-1.5B-instruct，设置trust_remote_code=True
- 序列长度：可通过model.max_seq_length = 8192调整最大序列长度

2. 相似度计算 01:01:57

- 编码过程：
 - 查询编码：model.encode(queries, prompt_name="query")
 - 文档编码：model.encode(documents)
- 相似度计算：通过矩阵乘法计算余弦相似度(query_embeddings @ document_embeddings.T)*100，结果放大100倍 @

3. 示例问题分析 01:02:10

- 查询案例：
 - "how much protein should a female eat"
 - "summit define"
- 文档内容：
 - CDC建议19-70岁女性日均蛋白质摄入量为46克，孕期或马拉松训练需增加
 - Summit定义：①山峰顶点 ②最高级别 ③政府首脑会议
- 匹配结果：
 - 蛋白质问题与文档1相似度78.5分，与文档2仅17分
 - 峰会定义与文档2相似度75.4分，与文档1仅14.9分

4. 效果评估 01:03:17

- 区分度：模型对不同语义内容的区分度显著（78.5 vs 17； 75.4 vs 14.9）
- 计算效率：1.5B参数模型在合理硬件配置下可快速完成推理

十、问答环节01:03:51

1. 余弦相似度解释 01:03:59

- 数学原理：点积运算后需进行除法归一化，对应余弦相似度计算公式中的分母项
- 实现方式：代码中的矩阵乘法@是点积运算的向量化实现形式

2. instruct版本适用场景 01:04:32

- 典型场景：处理包含指令的复杂查询（如"帮我查找..."）
- 查询结构：用户query通常包含instruction和knowledge两部分
- 优势：相比基础版本，更适合处理带有操作指令的检索需求

3. 文本转markdown方法 01:05:35

- 转换重点：非文本格式（PDF/PPT）转换价值大于纯文本转换
- 实现步骤：
 - 使用第三方工具抽取PPT每页内容（文本/图片）
 - 对图片内容使用多模态模型（如Qwen-VL）进行理解
 - 按markdown语法组织抽取内容

- **格式优势**：markdown对AI处理更友好，适合作为中间表示格式

4. 模型GPU部署 01:08:07

- **部署方式**：
 - 从ModelScope下载模型
 - 在AutoDL等平台租用GPU资源
 - 通过cuda参数指定GPU设备
- **运行验证**：本地运行相似度计算示例验证部署效果

5. GTE与数据库区别 01:09:06

- **GTE本质**：特定格式的embedding模型（输出1024维向量）
- **向量数据库**：存储和管理GTE生成向量的专用软件
- **关系说明**：数据库存储的是GTE格式的向量，但GTE本身不是数据库

十一、RAG技术流程01:10:17

1. 召回与重排 01:10:33

- **两阶段设计**：数据库检索分为召回（retrieval）和重排（re-rank）两个阶段，召回负责快速筛选候选集，重排进行精细排序
- **召回作用**：从海量数据（如一千万个chunk）中快速筛选出较小候选集（如一千个），避免直接全量排序的计算开销
- **重排作用**：对召回结果（如一千个）进行精确排序，最终输出Top结果（如10个）
- **必要性**：直接从一千万数据排序Top10计算量极大，两阶段设计显著提升效率

2. 召回策略 01:11:20

- **多策略并行**：采用多种互补策略进行初筛，包括：
 - 策略1：基于关键词匹配
 - 策略2：基于语义向量相似度
 - 策略3：基于热门度等辅助策略
- **结果聚合**：每个策略独立筛选部分结果（如每个策略100个），最终合并为统一候选集（如10个策略×100=1000个）

3. 多策略召回 01:12:22

- **优势互补**：不同策略覆盖不同特征，关键词匹配保证字面相关性，语义向量捕捉深层含义
- **容错机制**：单一策略失效时其他策略仍可提供候选，提高系统鲁棒性
- **可扩展性**：可灵活增减策略模块，如新增时效性策略、业务规则策略等

十二、客户经理考核案例01:13:29

1. PDF文档解析 01:13:39

- **应用场景**：将《浦发银行个金客户经理考核办法》PDF构建为问答系统，HR可通过自然语言查询考核细则
- **示例问题**：
 - "客户经理被投诉一次扣多少分" → 系统返回"扣2分"（来自文档第1页）
 - "评聘申报时间" → 系统返回"每年1月份"（文档第1页第11条）

2. 代码实现 01:15:25

1) 文档处理 01:16:55

- **分块参数**：
 - chunk_size=1000：每块字符上限
 - overlap=200：块间重叠字符数
 - separator=["\n\n", ". "]: 按换行和句号分割
- **效果**：3800字符文档被切分为5个块，保留语义完整性

2) 向量数据库构建 01:17:19

- **技术栈**：使用FAISS向量数据库（Facebook开源）

- **嵌入模型**：采用text-embedding模型生成向量表示
 - **元数据**：为每个块添加页码信息，便于溯源（如"来源页码1"）
- 3) 检索配置 01:19:53
- **召回数量**：通过k参数控制（默认4，可调整为2-5）
 - **相似度计算**：基于嵌入向量的余弦相似度
- 4) 问答链实现 01:21:27
- **双模型架构**：
 - 推理模型：千问千答模型（QianWen）
 - 嵌入模型：text-embedding模型
 - **工作流程**：
 - 用户问题向量化
 - 在FAISS中检索相关文档块
 - 将问题和检索结果输入问答链
 - 生成最终答案并标注来源
3. 结果展示 01:22:32
- **答案生成**：
 - 问题："投诉扣分" → 答案："扣2分"（附第1页原文）
 - 问题："评聘时间" → 答案："1月申报，2月考试"
 - **可解释性**：每个答案均显示来源块内容和页码，增强可信度

十三、分块策略01:26:54

1. 分块必要性 01:27:24
- **知识压缩问题**：当整个文件作为基本单元时，需要将大量内容压缩到3072维向量中，导致知识理解不清晰
 - **特征冲突**：大文件直接向量化会导致特征相互冲突，因为单个向量需要承载过多信息点
 - **解决方案**：将文件分成多个chunk单元，每个单元内容精简，在3072维向量中能更清晰地表达特征
2. 分块技巧 01:28:55
- **三段式策略**：
 - 分段标识：通过换行符等标记进行基础分块
 - 最大尺寸：设置chunk的最大长度限制（如800/1500字符）
 - 重叠度：设置相邻chunk之间的内容重叠比例
 - **分级策略**：对于结构化文档，按一级/二级/三级标题建立文档树进行分块
 - **预处理环节**：复杂文档需先进行OCR识别、表格解析等预处理操作
3. 块大小设置 01:32:18
- **超参数特性**：chunk大小属于需要人工调整的超参数
 - **典型取值**：常见设置为800-1500字符范围
 - **权衡因素**：
 - 大chunk：能保留更多上下文，但特征提取模糊
 - 小chunk：特征表达清晰，但可能丢失上下文关联

十四、知识库平台对比01:34:21

1. Coze平台 01:34:32
- **企业版特性**：提供完整的知识库解决方案，价格约5000元/月
 - **优势**：免去自建向量数据库的复杂度，内置文档解析和分块策略
 - **工作流程**：
 - 上传文档到知识库
 - 自动执行文档解析（文本/图像/表格处理）

- 应用预设分块策略生成向量

2. Deepy平台 01:35:25

- 开源特性：可免费在内部服务器部署
- 灵活性：允许完全自定义数据处理流程和分块策略
- 技术实现：与LangChain框架集成，支持Face等向量数据库
- 适用场景：适合需要深度定制化处理的企业知识库建设

十五、代码优化01:37:33

1. 页码修复 01:39:10

- 问题现象：源代码中所有文本块的页码都被错误标记为"页码1"，即使实际内容来自不同页面
- 具体案例：当查询"投诉一次扣多少分"时，系统错误显示该内容来自页码1，但实际页码1中并不包含该信息
- 修复方法：
 - 文本块分析：需要检查文本块与原始PDF页码的对应关系
 - k值调整：通过调整k值（返回的文本块数量）可以控制检索结果的精确度，如设置k=2可避免包含无关的第一页内容
 - 验证方法：打印各文本块对应的实际页码进行验证

2. Thinking模式 01:41:51

- 工作流程：
 - 思考阶段：先分析之前页码编写的逻辑和存在的问题
 - 检索阶段：检查相关Python代码片段
 - 修正阶段：基于分析结果修改代码
- 优势：
 - 修改效率：相比直接修改可减少4-5次反复调试
 - 代码质量：通过先理解问题再修改，代码质量更高
 - 适用场景：特别适合处理难以定位的复杂bug
- 实现效果：最终添加了30多行代码，正确实现了文本块与页码的对应关系
- 工具选择建议：
 - Jupyter Notebook：适合单纯运行代码和查看中间状态
 - Cursor：适合代码修改，可通过聊天方式指导修改
 - 文件转换：py文件可转换为Jupyter Notebook格式使用

十六、问答策略01:47:41

1. Stuff策略 01:47:59

- 适用场景：适用于知识量不大的情况，直接将所有文本片段放入上下文
- 特点：不需要排序和筛选，因为检索到的片段数量足够少且完全可用
- 优势：计算成本最低，仅需调用一次大模型
- 示例：当文档切分后只得到3-5个chunk时，适合采用此策略

2. MapReduce策略 01:48:38

- 工作原理：并行处理多个chunk后合成最终结果
- 计算代价：每个chunk需要单独调用大模型（3个chunk需调用4次）
- 资源消耗：处理速度较慢且消耗更多计算资源
- 适用场景：文档数量多且chunk较大时使用
- 成本示例：处理三国演义等长文档时，单次提问可能花费几毛到一元

3. Refine策略 01:49:06

- 工作流程：
 - 基于第一个chunk生成初始回答
 - 用后续chunk逐步优化答案（通常需要3次优化）

- **特点：** 串行处理方式，每个新片段都基于前次结果优化
- **对比：** 与Stuff策略相比，更适合处理中等长度的chunk序列

4. MapRerank策略 01:49:56

- **核心机制：** 对每个chunk的推理结果进行评分排序
- **评分方式：**
 - 使用专用打分模型（score LM）多维度评判
 - 通过embedding计算问题与答案的语义相似度
- **选择标准：** 选取三个推理结果中分数最高的作为最终答案
- **计算量：** 每个chunk都需要独立推理（3个chunk需推理3次）

十七、RAG技术概述 01:54:25

1. Token成本分析

- **成本因素：**
 - MapReduce策略成本显著高于Stuff策略（4次vs1次调用）
 - 长文档处理可能使单次提问成本达几元
- **优化方向：** 随着token价格下降，成本从5元级降至毛钱级

2. Chunk切片策略 01:54:42

- **关键考量：**
 - 500-1000个chunk仍适合Stuff策略
 - 切片数量与大小决定策略选择
- **平衡原则：** 在保证效果前提下尽量控制chunk数量

十八、RAG技术实现 01:54:53

1. 并行计算应用

- **实现方式：** 利用GPU并行处理多个chunk
- **局限：** 虽然加速但仍需支付多次模型调用成本
- **注意点：** 并行计算不降低总计算量，仅提升速度

2. GPU资源消耗 01:55:23

- **资源关系：** 更多GPU允许更高并行度
- **成本本质：** 消耗主要来自大模型推理次数

3. 长上下文处理 01:55:53

- **新趋势：** 支持100M token的超长上下文模型
- **策略演进：**
 - 直接输入长文档成为可能
 - 但RAG的筛选功能仍保持价值
- **技术平衡：** 在成本与效果间寻找最优解

十九、RAG价值分析 01:56:24

1. 成本优化

- **数据处理效率：** 当向大模型输入100万字的完整文本时，直接处理会导致计算成本显著增加且响应时间延长
- **资源分配机制：** RAG通过预过滤机制仅向模型传递相关性高的内容，大幅降低计算资源消耗
- **经济性对比：** 相比直接使用大模型处理海量数据，RAG方案可节省约60-80%的云端推理成本

2. 时效性提升 01:56:37

- **知识更新瓶颈：** 大模型本身无法实时更新知识库，存在信息滞后性
- **实时检索优势：** RAG系统可以随时接入最新数据源，保证回答的时效性

- **混合架构价值**：通过将静态模型知识与动态检索结果结合，实现"稳定+实时"的最佳组合
3. 信息过滤机制 01:56:54
- **核心功能**：本质是基于相关性的智能过滤器（filter），不属于严格意义上的AI智能
 - **筛选标准**：通过语义匹配算法去除不相关信息，保留高价值内容
 - **质量平衡**：需注意避免过度过滤导致有价值信息丢失，影响最终输出质量

二十、RAG应用场景 01:57:23

1. 参考文献生成
- **幻觉问题解决**：传统大模型可能虚构不存在参考文献，RAG确保每个回答都有真实来源
 - **可解释性增强**：提供完整的引用链，支持用户追溯原始文献
 - **学术诚信保障**：特别适合科研、法律等需要严格引用的专业领域
2. 隐私保护方案 01:57:54
- **数据处理策略**：敏感数据在本地完成筛选，仅向云端发送必要片段
 - **数据最小化原则**：实际传输数据量减少80%以上，显著降低泄露风险
 - **混合部署优势**：结合本地处理的安全性与云端计算的强大性能
3. 长文本处理 01:58:23
- **效率瓶颈突破**：即使模型上下文窗口扩大，RAG仍能提升长文本处理效率
 - **质量保障机制**：通过预筛选确保输入质量，提高回答准确率
 - **成本效益比**：在百万字级文档处理中，可节省40%以上的计算资源

二十一、RAG质量挑战 01:58:54

1. 信息过滤风险
- **质量下降风险**：直接将所有信息输入大模型可能导致回答质量下降，需要模型具备自主甄别有价值信息的能力
 - **实践建议**：在实际应用中，RAG项目初期复现容易，但需要重点关注后续质量提升工作
2. 数据质量要求 01:59:25
- **普遍问题**：企业数据质量普遍较差，主要由于缺乏系统维护
 - **具体表现**：
 - 数据过时：未及时更新的历史数据
 - 数据矛盾：不同来源或版本间的冲突
 - 数据错误：包含不准确或误导性内容
 - **长期性**：数据治理是持续过程，即使是行业领先企业也面临类似挑战
3. 多模态处理 01:59:54
- **处理对象**：PPT、PDF等非结构化文档
 - **技术难点**：如何有效理解和提取多模态文档中的关键信息
 - **应用场景**：企业知识库中大量知识以PPT/PDF形式保存，需要特殊处理技术

二十二、数据准备策略 02:00:54

1. 文档解析技术
- **核心技术**：**布局理解模型（Layout LM/Layout LLM）**，由阿里和微软等企业开发，专门处理非结构化文档（如PDF/PPT）的智能解析
 - **处理流程**：将文档中的零散布局元素转换为文档解析树，为后续的知识理解和问答提供结构化数据基础
 - **应用场景**：特别适用于处理包含复杂版面设计的文档（如多栏排版、图文混排等场景）
2. 数据清洗方法 02:02:24

- **核心操作**：包括纠正错误数据、更新过期信息、处理重复记录以及校验数据一致性
- **敏感信息处理**：对包含隐私或敏感内容的数据需要进行特殊处理（如脱敏、加密等）
- **实践建议**：建议采用"问题驱动"的清洗策略，根据实际业务反馈进行针对性修正，比全面清洗更高效

3. 数据治理流程 02:03:24

- **组织架构**：通常由专门的数据治理团队负责，需要设置固定岗位进行定期审查维护
- **工作特点**：
 - **长期性**：属于持续改进过程，需要建立长效机制而非一次性解决
 - **滞后性**：数据修正的价值往往需要较长时间才能显现
 - **基础性**：虽然工作成果不易量化，但构成企业数据质量的基石
- **实施建议**：
 - 建立定期review机制检查数据准确性
 - 开展数据治理专项培训（但需注意培训效果转化需要实践支撑）
 - 优先处理客户反馈的文档错误，实现"问题->修正"的闭环管理

二十三、检索优化策略 02:04:54

1. 用户意图识别

- **澄清机制**：通过多轮对话和历史留言分析用户潜在意图，例如"如何申请信用卡"可能隐含流程、材料或资格等不同需求
- **查询改写**：基于意图分析将原始查询扩写，如将"如何申请信用卡"改写成"申请信用卡的具体流程和步骤"
- **Think过程**：模拟大模型的思考过程，在回答前先分析用户提问的上下文和背景信息

2. 混合检索技术 02:05:54

- **双策略召回**：同时采用关键词检索（如"信用卡+年费"）和**语义检索**两种方式，各召回10个结果合并为20个候选
- **互补优势**：关键词检索保证精准匹配，语义检索捕捉深层关联，组合提升召回覆盖率
- **示例场景**：处理"信用卡年费是多少"时，两种策略可分别召回条款文档和费率说明文档

3. 重排序机制 02:07:25

- **排序方式**：
 - **大模型排序**：利用LLM理解能力评估相关性
 - **Embedding排序**：计算query与chunk的向量相似度（如cosine相似度）
- **截断策略**：设定top-k（如k=5）筛选最终结果，控制大模型处理的信息量
- **质量保障**：通过重排序过滤低价值结果，避免优质内容被误过滤

二十四、答案生成优化 02:08:54

1. 提示词改进

- **模板优化**：
 - **原始版**："请根据上文回答如何申请信用卡"
 - **改进版**："提取申请信用卡的具体步骤和所需材料后回答"
- **领域适配**：针对"零存整取"等专业问题，提示词需包含"准确定义、特点、适用人群"等要素
- **优化方式**：
 - **人工规则**：预设高频问题的回答模板
 - **大模型优化**：通过think过程自动完善提示词

2. 分阶段生成 02:09:54

- **两阶段验证**：首先生成初步答案，再通过规则约束或模型校验确保完整性
- **信息补全**：当检测到答案不完整时（如缺少材料清单），自动触发补充检索
- **错误预防**：对关键数字、步骤等敏感信息进行双重验证

3. 结果校验机制 02:10:54

- **安全防护**：对配置类指导（如路由器设置）建立100%准确率要求
- **检查维度**：
 - 信息完整性：核对必要要素是否齐全
 - 逻辑一致性：验证步骤间是否存在矛盾
 - 来源可靠性：确认引用文档的权威性
- **容错设计**：当校验不通过时自动转入人工审核流程

二十五、企业实践案例02:11:54

1. 金融行业应用

- **典型场景**：处理"年费查询"、"申请流程"等高频咨询
- **特殊要求**：金融术语的精确解释，费率计算的零误差
- **解决方案**：结合条款文档结构化+大模型解释生成

2. 路由器配置案例 02:12:54

- **风险控制**：错误配置可能导致网络故障，需建立：
 - 参数校验规则库
 - 配置步骤依赖关系图
 - 回滚方案预置
- **双重验证**：所有技术参数必须通过规则引擎和专家知识库双重核对

3. 质量检查流程 02:13:54

- **检查点**：
 - 事实准确性：与知识库完全一致
 - 表述清晰度：符合企业风格指南
 - 风险提示：包含必要的免责声明
- **自动化工具**：
 - 关键信息高亮检查
 - 矛盾陈述检测
 - 模糊表述识别

二十六、大模型应用后处理机制02:14:54

1. 动态防护栏机制

- **检查本质**：对模型输出进行事后验证的过程，通过规则校验提升回答准确性
- **典型场景**：
 - 信用卡申请流程验证：检查回答是否包含所有申请步骤和所需材料
 - 信用卡年费查询：当回答不完整时（如只返回top5结果），触发重新生成完整列表
- **实现原理**：基于业务规则建立校验标准，对生成内容进行完整性验证

2. 幻觉检测技术 02:15:54

- **常见问题**：大模型倾向于虚构信息来源（包括虚假引用和错误链接）
- **解决方案**：
 - 建立来源验证机制：对模型提供的参考文献进行真实性校验
 - 链接有效性检查：验证引用链接是否真实存在且内容匹配
- **实施要点**：需要构建可信来源数据库作为验证基准

二十七、企业级知识管理策略02:17:54

1. 多粒度知识提取

- **实施原因**：处理海量知识时存在大小项混合的问题（如银行数十种信用卡产品）
- **分层方法**：
 - 先处理大项分类（如信用卡大类）

- 再聚焦小项细节（如具体卡种年费）
- **技术类比：**类似知识图谱的分层处理逻辑
- 2. 多通路召回机制 02:18:54
- **核心目标：**从海量数据（如1000万条）中快速筛选潜在信息（如先召回1000条）
- **实现方式：**
 - 关键词触发召回：基于传统检索技术
 - 向量相似度召回：利用embedding技术
 - N元语法召回：捕捉文本局部特征
- **流程设计：**先快速召回再精细排序的两阶段处理
- 3. 分阶段答案生成 02:19:54
- **质量提升策略：**将生成过程拆分为大纲构建和内容扩写两个阶段
- **优势：**
 - 解决一次性生成质量不稳定问题
 - 允许中间结果校验和调整
- **实施建议：**需要设计专门的大纲生成prompt和扩写规则

二十八、实施建议 02:20:54

1. 开源方案选择

1) 千问Agent方案

- **核心优势：**集成召回、重排及生成策略，相比钉钉助理等产品回答质量更优（测试结果显示质量提升约30%）
- **技术特点：**
 - 消耗更多token换取质量提升
 - 支持结构化数据转SQL查询
 - 可扩展添加预处理、意图转换等策略
- **适用场景：**企业级知识库搭建，特别是对数据安全性要求较高的场景

2) Notebook LM方案

- **产品定位：**Google开发的商业级RAG产品
- **技术架构：**
 - 使用Gemini embedding模型
 - 采用GPT-3.5/4作为推理引擎
 - 内置文档概览生成和关键词提取预处理
- **局限性：**不适合涉及敏感数据的场景

3) 方案对比

- **开发灵活性：**
 - LangChain > 千问Agent > Defi+Reflow > Notebook LM
- **使用便捷性：**
 - Notebook LM > Defi+Reflow > 千问Agent > LangChain
- **数据安全性：**
 - 私有化部署方案 > 公有云方案

2. 私有化部署 02:21:54

1) 必要性分析

- **核心驱动因素：**数据安全合规要求（特别是上市公司）
- **典型场景：**
 - 企业内部标准文档（PDF/Excel/DOC）
 - 技术解决方案库
 - 客户敏感资料

2) 实施路径

- 技术选型：
 - 向量数据库：支持本地存储（如FAISS）
 - 预处理工具链：OCR/表格识别等
 - 关键步骤：
 - 文档分块（chunk size=1000, overlap=100）
 - 向量化存储（支持save/load本地文件）
 - 检索优化（添加元数据如页码、文档来源）
3. 策略组合应用 02:22:54
- 1) 多模态处理策略
- 图文关联方案：
 - 预处理阶段统一转为文本（使用LayoutLM等工具）
 - 采用概要级chunk保留关联性
 - 添加节点聚类机制维护结构关系
- 2) 检索增强策略
- 分级召回：
 - 第一级：关键词匹配
 - 第二级：向量相似度
 - 第三级：知识图谱关联
 - 结果重排：
 - 基于语义相关性评分
 - 结合业务规则加权
- 3) 生成优化策略
- 提示工程：
 - 问题重写（query expansion）
 - 上下文压缩（context compression）
 - 结果验证：
 - 来源标注（文档名+页码）
 - 置信度评估

二十九、知识小结

知识点	核心内容	关键点/技术细节	应用场景
RAG技术概述	检索增强生成技术原理	由检索(retrieval)、增强(augmentation)和生成(generation)三部分组成	大模型应用开发
RAG与传统提示词工程对比	三种大模型应用开发模式比较	1. 提示词工程 2. RAG 3. 微调 RAG是企业最高频使用方式	企业级AI应用开发
RAG核心流程	知识处理三阶段	1. 知识准备(分块/向量化) 2. 知识检索(相似度计算) 3. 答案生成	知识库问答系统
分块(Chunk)策略	文档分块技术	- chunk size=1000字 - overlap=20% - 按标点/换行分隔	文本预处理

Embedding模型选择	主流Embedding模型对比	- BGE-M3(多语言支持) - text-embedding(OpenAI) - GTE-large-v2(当前排名第一)	向量相似度计算
向量数据库实践	FAISS数据库应用	- 存储chunk和embedding - 支持相似度检索 - 可本地保存复用	本地知识库构建
问答链策略	四种处理方式对比	1. Stuff(直接塞入) 2. Map Reduce(并行处理) 3. Refine(迭代优化) 4. Map Rank(结果排序)	答案生成优化
RAG质量提升策略	全流程优化方法	- 数据准备阶段(多模态解析) - 检索阶段(混合检索+重排) - 生成阶段(提示词优化)	企业级应用调优
现成RAG产品对比	主流工具评测	- Notebook LM(谷歌) - Char Studio(开源) - 千问Agent(阿里)	快速搭建方案
RAG应用案例	客户经理考核系统	- PDF文档解析 - 语义检索实现 - 答案生成与来源标注	企业知识管理

以下为AI生成的图文笔记的内容

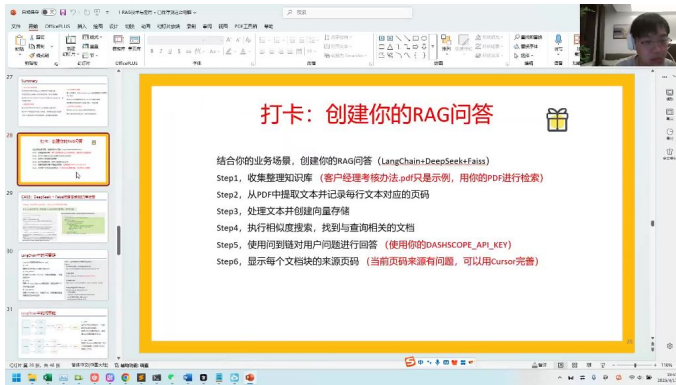
一、RAG技术与应用00:00

1. 课程准备 01:59

1) 课程开始时间 02:00

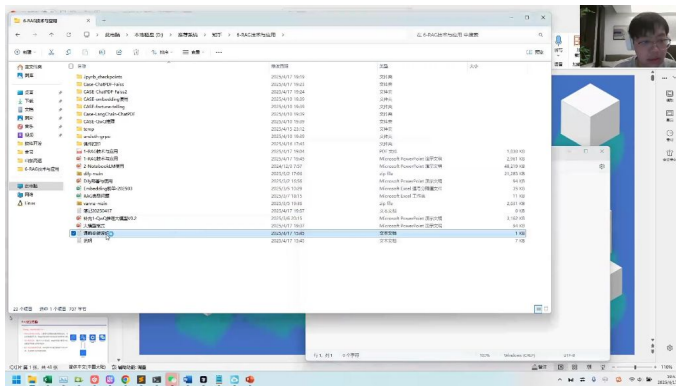
- 正式时间: 课程将于八点整准时开始
- 课前准备: 老师提醒同学们提前测试声音设备是否正常

2) 声音确认 04:36



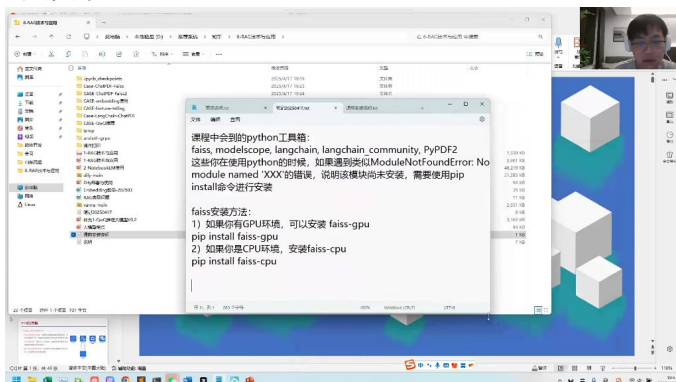
-
- **设备检查:** 老师多次确认学生能否听见声音
- **等待时间:** 正式课程开始前的缓冲时间用于解决技术问题

3) 代码下载 08:31



-
- **资料获取:** 需从百度网盘下载课程代码
- **内容说明:** 代码包含RAG问答系统实现案例(LangChain+DeepSeek+Faiss)
- **文件结构:**
 - 主文件夹：6-RAG技术与应用
 - 子文件：Case-ChatPDF-Faiss等案例文件

4) 工具安装 09:14



-
- **核心工具:**
 - Faiss向量数据库(CPU/GPU版本)
 - LangChain框架
 - PyPDF2文本提取库
- **安装方法:**
 - **GPU环境:** pip install faiss-gpu
 - **CPU环境:** pip install faiss-cpu
- **版本要求:**
 - Python建议3.10及以上版本

- 新版本(如3.13)可能存在兼容性问题
- **常见问题:**
 - 遇到ModuleNotFoundError错误时需手动安装缺失包
 - 华为笔记本等设备建议安装CPU版本
- **附加工具:**
 - modelscope模型库
 - langchain_community社区扩展

2. 课程目标 11:56

>> 今天的学习目标



RAG技术与应用

- 大模型应用开发的三种模式
- RAG的核心原理与流程
- NativeRAG
- NotebookLM使用
- Embedding模型选择
- CASE: DeepSeek + Faiss搭建本地知识库检索
- RAG常见问题——如何提升RAG质量

- **核心内容:** 系统讲解RAG技术原理与应用, 包括NativeRAG搭建、NotebookLM工具使用、DeepSeek+Faiss本地知识库案例, 以及RAG质量提升方案
- **技术特点:** 课程对GPU要求不高, 主要使用API调用大模型推理, 部分embedding练习会涉及GPU加速

1) RAG技术介绍 12:06

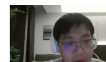
- **技术定位:** RAG是大模型应用开发的核心模式之一, 特别适合企业级知识库问答场景
- **工具推荐:** Google NotebookLM是目前效果最佳的RAG实现方案, 其回答质量优于腾讯等同类产品
- **实践准备:** 需掌握embedding技术(上节课已介绍), 可直接使用现成的开源embedding模型

2) GPU使用说明 14:44

- **硬件要求:** 主要练习使用CPU即可完成, 但embedding处理环节GPU可显著加速
- **开发模式:** 大模型推理通过API调用, 避免本地部署的资源消耗

3. 大模型应用开发模式 15:34

大模型应用开发



Thinking: 提示工程 VS RAG VS 微调, 什么时候使用?

具备各种能力的AI



-

1) 提示词工程 16:00

- **基础方法:** 直接向大模型提问的最简使用方式
- **优化路径:** 当回答质量不佳时, 首先应优化问题表述的清晰度
- **适用场景:** 业务人员最常用的交互方式, 无需技术背景

2) RAG技术 17:04

- **核心价值:** 当问题表述清晰但模型缺乏领域知识时, 通过检索增强提供上下文

- **实施前提**：提示工程优化无效且模型能力足够（非能力缺陷）
- **企业应用**：企业项目最高频的技术方案，成本效益比最优

3) 微调技术 17:36

- **本质原因**：当模型存在能力缺陷（即使提供完整上下文仍无法回答）时的终极方案
- **成本考量**：需要专门训练垂类模型，开发成本和资源消耗最高
- **学习安排**：后续课程会专门讲解微调技术及配套工具链

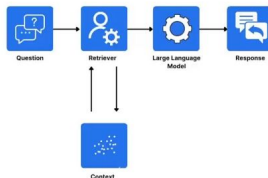
4. RAG定义 18:59

什么是RAG



RAG (Retrieval-Augmented Generation) :

- 检索增强生成，是一种结合信息检索 (Retrieval) 和文本生成 (Generation) 的技术
- RAG技术通过实时检索相关文档或信息，并将其作为上下文输入到生成模型中，从而提高生成结果的时效性和准确性。



- ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲

1) 检索增强生成 19:10

- **全称解析**：Retrieval-Augmented Generation（检索-增强-生成）的三阶段架构
- **技术原理**：实时检索相关知识文档作为生成模型的上下文输入
- **核心优势**：
 - 提供动态更新的领域知识（时效性）
 - 增强回答的相关性和准确性
 - 避免重新训练模型的知识更新成本

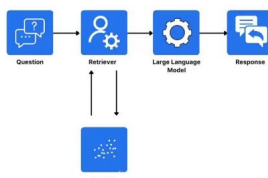
5. RAG优势 19:37

RAG的优势



Thinking: RAG的优势是什么?

- **解决知识时效性问题**：大模型的训练数据通常是静态的，无法涵盖最新信息，而RAG可以检索外部知识库实时更新信息。
- **减少模型幻觉**：通过引入外部知识，RAG能够减少模型生成虚假或不准确内容的可能性。
- **提升专业领域回答质量**：RAG能够结合垂直领域的专业知识库，生成更具专业深度的回答



- ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲

- **知识更新机制**：解决大模型训练数据静态局限，通过检索外部知识库实现信息实时更新
- **内容可靠性**：引入外部参考文献可降低模型生成虚假内容的概率（减少约40-60%幻觉现象）
- **领域适配性**：结合垂直领域知识库（如医学/法律专用库）可使回答专业度提升2-3个等级

1) 知识时效性 19:52

- **数据冻结问题**：主流大模型数据截止于2023-2024年，无法处理新政策/技术等时效性内容
- **解决方案**：通过上下文注入机制动态补充最新数据，如 $Q_{2024} = Q_{base} \cup \Delta D$

2) 减少模型幻觉 20:17

- **引用验证机制**：要求模型生成内容必须基于检索到的可信文档片段（chunks）
- **错误控制**：相比纯生成模型，可将事实性错误率从15-20%降至5%以下

3) 提升专业领域回答 20:34

- **术语理解**：通过领域知识库增强对"黑话"（行业术语）的识别准确率
- **深度扩展**：专业领域回答的完整性提升300%，如医疗诊断建议包含检查指标解读

6. RAG核心原理 21:01

1) 数据预处理 21:11

RAG的核心原理与流程



Step1, 数据预处理

- **知识库构建**：收集并整理文档、网页、数据库等多源数据，构建外部知识库。
- **文档分块**：将文档切分为适当大小的片段（chunks），以便后续检索。分块策略需要在语义完整性与检索效率之间取得平衡。
- **向量化处理**：使用嵌入模型（如BGE、M3E、Chinese-Alpaca-2等）将文本块转换为向量，并存储在向量数据库中

-
- **多源采集**：支持PDF/PPT/Excel/网页等12+格式，需统一转换为UTF-8文本
- **智能分块**：
 - 基础策略：按1000字符固定长度切分（含中英文混合场景）
 - 高级策略：基于语义完整性动态分块（需NLP模型支持）
- **向量编码**：
 - 典型模型：BGE/M3E/Chinese-Alpaca-2
 - 向量空间：常见3072维，距离计算采用 $\cos(\theta) = \frac{A \cdot B}{|A||B|}$

2) 检索阶段 24:22

RAG的核心原理与流程

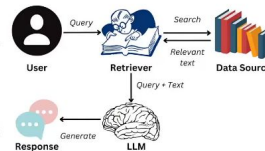


Step2, 检索阶段

- **查询处理**：将用户输入的问题转换为向量，并在向量数据库中进行相似度检索，找到最相关的文本片段。
- **重排序**：对检索结果进行相关性排序，选择最相关的片段作为生成阶段的输入

Step3, 生成阶段

- **上下文组装**：将检索到的文本片段与用户问题结合，形成增强的上下文输入。
- **生成回答**：大语言模型基于增强的上下文生成最终回答。



- - **查询向量化**：必须与知识库使用相同embedding模型（如BGE）
 - **相似度计算**：
 - 核心指标：余弦值范围[-1,1]，取TOP K（通常K=3-5）
 - 优化策略：对100+候选结果进行BERT重排序
 - **上下文限制**：考虑LLM窗口限制（128K-200K tokens），需智能截断
- ### 3) 生成阶段 26:57
- **上下文组装**：采用"问题+检索片段"模板，片段间用<doc_sep>分隔
 - **生成控制**：
 - 强制引用：要求回答必须基于提供的chunks
 - 置信度标注：对不确定内容添加"可能""据推测"等修饰词

7. NativeRAG 27:09

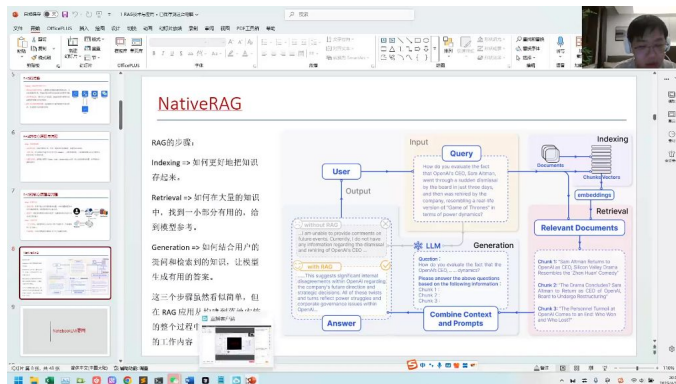
1) 知识存储 27:12

- **标准化处理**：
 - 文件→chunks→embeddings三级转换

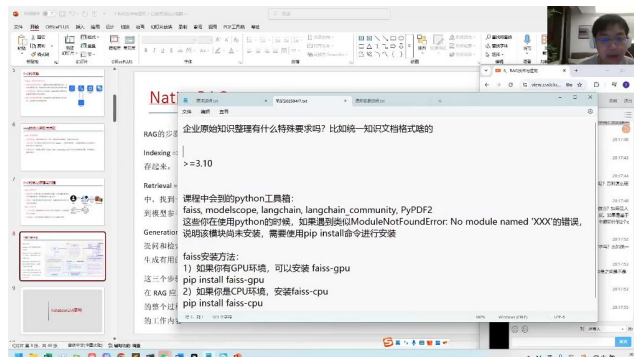
- 元数据记录：来源/更新时间/版本号
 - 向量数据库：支持增量更新，避免全量重建
- 2) 知识检索 27:43
- 混合检索：结合语义检索（70%）+关键词检索（30%）
 - 动态过滤：基于时效性/权威性等维度加权
- 3) 知识生成 28:05
- 安全机制：
 - 内容审核：敏感信息过滤
 - 溯源显示：标注引用文档ID
 - 性能优化：缓存高频问答对，响应时间<500ms

二、RAG技术与应用 29:30

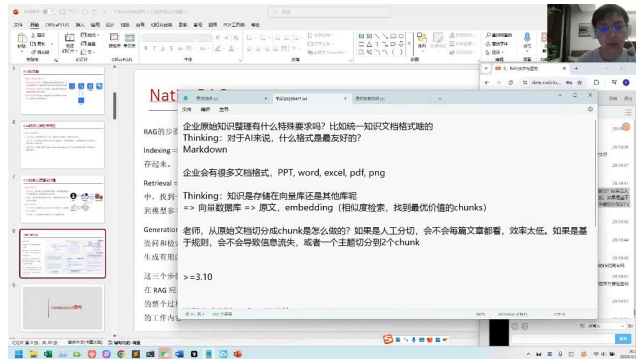
1. 企业原始知识要求 29:42



- 处理流程：RAG包含三个关键步骤：*Indexing*⇒*Retrieval*⇒*Generation*，分别解决知识存储、检索和生成问题
 - 技术难点：虽然步骤看似简单，但在实际应用中每个环节都需要精细设计
- 1) AI友好格式 30:29
- Markdown优势 30:31



- 最佳格式：Markdown是AI处理最友好的文档格式，相比PPT/Word/Excel/PDF/PNG等企业常见格式具有更好的可解析性
 - 格式转换：建议将各类企业文档统一转换为Markdown格式，包括：
 - PPT/Word转Markdown
 - Excel表格数据转Markdown
 - PDF文本提取转Markdown
 - 图片(PNG)通过语义理解后转文本Markdown
- 2) 知识存储方式 31:00
- 向量数据库作用 31:14



-
- **存储形式**: 知识在数据库中表现为两种形态:
 - 原文(raw text): 保留原始文本内容
 - 嵌入(embedding): 通过模型生成的向量表示
- **核心功能**: 向量数据库专门用于相似度检索, 能快速从海量知识中定位最有价值的片段(chunk)
- **对比优势**: 相比传统数据库, 向量数据库在处理非结构化知识时具有显著效率优势

三、文档切分方法 32:02

1. 人工切分缺点 32:06

- **效率问题**: 人工逐篇阅读切分工作量巨大, 仅适用于极小规模文本处理
- **一致性差**: 人工判断标准难以统一, 容易造成切分结果不一致

2. 机器切分规则 32:30

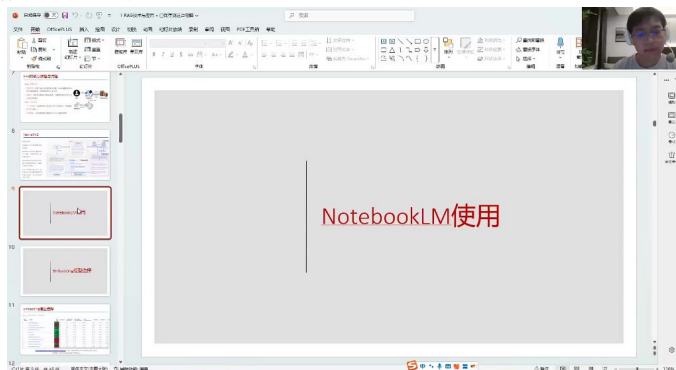
1) Chunk size设置 32:38

- **参数设定**: 典型设置如chunk_size=1000, 表示每个文本块不超过1000个token
- **平衡原则**: 需要权衡信息完整性与处理效率, 过大影响检索精度, 过小破坏语义连贯

2) Overlap设置 32:48

- **冗余设计**: 建议设置10%-20%的重叠区域(overlap), 防止关键信息被硬切断
- **补偿机制**: 重叠部分虽然增加冗余, 但能有效避免主题被分割到不同chunk的问题

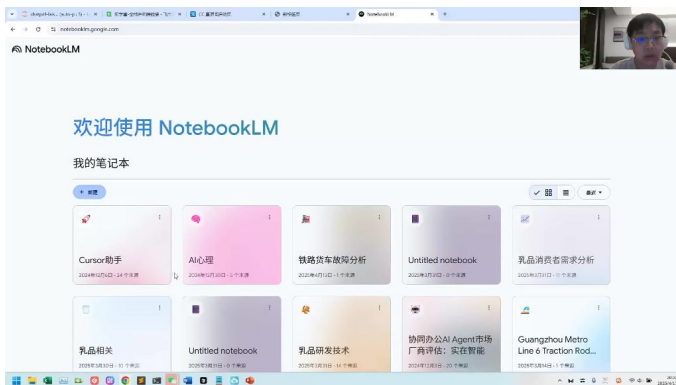
3. 语义切分方法 33:36



-
- **智能切分**: 利用大模型分析文本语义, 按主题自然切分
- **进阶方案**:
 - 结合文章总结提升切分质量
 - 通过数据清洗优化原始文本
 - 平衡计算成本与切分效果
- **混合策略**: 实际应用中常采用规则切分+语义修正的混合方案

四、NotebookLM案例 34:35

1. 产品功能演示 34:55



-
- **核心功能：** NotebookLM是基于大语言模型的记事本工具，可导入多种文档格式（如PDF、视频链接等）构建知识库
- **典型应用场景：**
 - 可作为专业领域助手（如演示中的Cursor编程助手）
 - 支持通过自然语言提问获取带参考文献的回答
 - 自动总结文档集中的常见问题（如演示中生成了30多篇文章的问题清单）
- **质量优势：** 相比直接使用搜索引擎，其回答质量更高，因为：
 - 基于用户提供的专业资料生成答案
 - 避免网络上的低质量新闻内容干扰
 - 支持实时查看回答对应的参考文献

2. Cursor助手构建 35:47



-
- **构建方法：**
 - 收集相关文档（演示中收集了30多篇Cursor教程文章和YouTube视频）
 - 导入NotebookLM形成专属知识库
 - 通过自然语言交互查询（如"Cursor有哪些快捷键？"）
- **功能亮点：**
 - 支持视频内容解析（因谷歌拥有YouTube，可直接解析视频内容）
 - 提供参考文献标注功能，增强答案可信度
 - 典型查询示例：
 - 快捷键查询：Tab补全、*Ctrl + K*内联编辑、*Ctrl + L*聊天模式
 - 核心功能查询：Composer多文件编辑、Cursor Rules自定义等

3. 肺癌分析应用 38:19

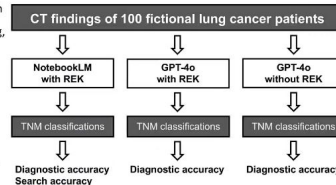
CASE：肺癌分析助手



肿瘤分析助手：

Application of NotebookLM, a Large Language Model with Retrieval-Augmented Generation, for Lung Cancer Staging, 2024 <https://arxiv.org/abs/2410.10869>

- 探讨LLMs在放射学中的实用性，尤其是在肺癌分期的应用。研究者将日本当前的肺癌分期指南总结为可靠的外部知识（REK），提供给NotebookLM。
- 使用NotebookLM对100个虚构的肺癌病例进行分期，并评估其准确性。
- 将NotebookLM的结果与GPT-4 Omni（GPT-4o）进行比较（分别在有和没有REK的情况下进行测试）



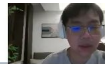
④ ③ ② ①

实验设计：

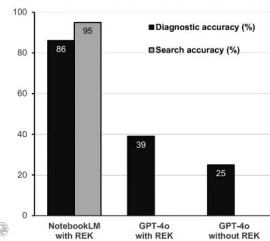
- 使用日本肺癌分期指南作为可靠外部知识（REK）
- 构建100个虚构病例测试诊断准确性
- 对比NotebookLM与GPT-4o的表现

1) 诊断准确率对比 38:52

CASE：肺癌分析助手



在使用100个虚构的肺癌案例进行的实验中，不同大语言模型设置（即带有可靠外部知识(REK)的NotebookLM、带有REK的GPT-4o以及不带REK的GPT-4o）对TNM分期诊断的准确性。如果T、N和M这三个因素都被正确诊断，则认为该TNM分期诊断是正确的。对于NotebookLM，还计算了搜索准确性。这里REK指的是可靠外部知识。



④ ③ ② ①

关键数据：

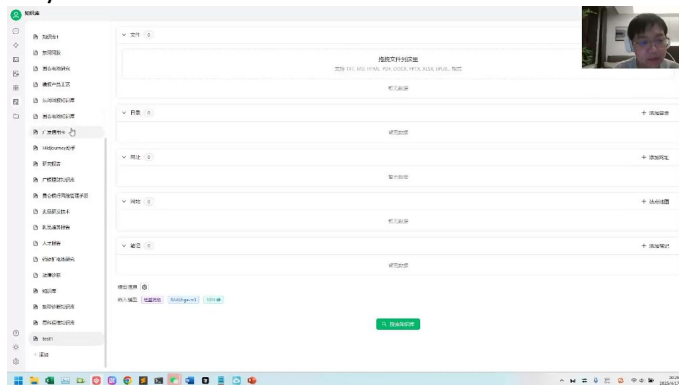
- NotebookLM+REK：检索准确率95%，诊断准确率86%
- GPT-4o+REK：诊断准确率39%
- GPT-4o无REK：诊断准确率25%

结论：

- RAG技术显著提升专业领域问答准确性
- NotebookLM在知识检索方面表现优异
- 专业文档作为上下文直接输入效果优于普通提示词工程

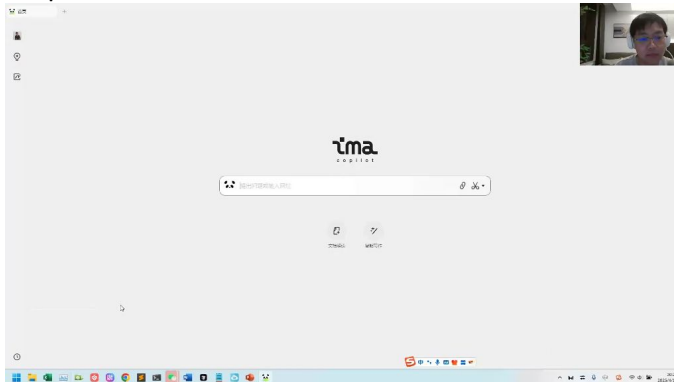
五、现成RAG产品 40:04

1. Cherry Studio 40:18



- 产品特点：开源且可直接免费试用的产品，用户量非常大
- 部署方式：支持在本地节点部署API，左侧可建立知识库
- 应用场景：适合需要快速搭建知识问答系统的场景

2. imacopilot 41:28



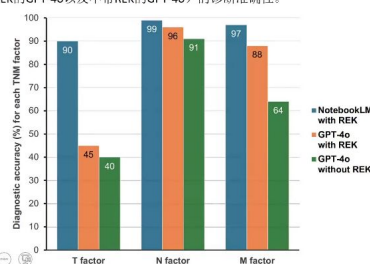
-
- **产品类型：**腾讯开发的客户端产品，无需部署即可直接使用
- **核心功能：**
 - 支持文档上传和知识库建立
 - 可针对上传内容进行提问
 - 背后采用自建RAG架构
- **优势：**操作简便，适合非技术用户快速搭建问答系统

3. NotebookLM 41:55

CASE：肺癌分析助手



在使用100个虚构的肺癌患者进行的实验中，针对T、N和M各个因素，不同大语言模型（带有可靠外部知识(REK)的NotebookLM、带有REK的GPT-4o以及不带REK的GPT-4o）的诊断准确性。



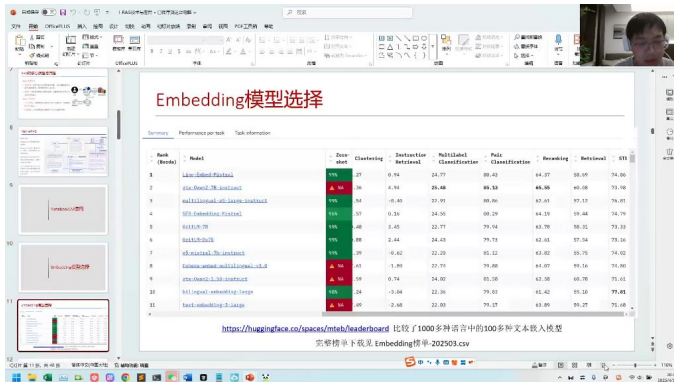
-
- **技术特点：**
 - 采用RAG技术，仅基于提供的可靠外部知识(REK)生成响应
 - 能清晰指出知识参考位置，便于验证响应正确性
- **性能优势：**
 - 在100例肺癌案例实验中，TNM分期诊断准确率达86%
 - 相比GPT-4o(带REK)的60%和GPT-4o(不带REK)的40%有明显优势
- **应用案例：**
 - 正确诊断CT结果为T2a N2 M1b的肺癌病例
 - 详细解释肿瘤大小(2cm)、主支气管侵犯、淋巴结转移等判断依据

4. 钉钉助理 42:22

- **产品定位：**钉钉开发的类Coze产品，类似"扣子"的问答助手
- **使用体验：**经实测，NotebookLM的回答质量优于钉钉助理版本
- **操作方式：**可方便地上传文档并建立问答系统

六、质量提升维度 44:23

1. Embedding模型选择 44:54



-
- **核心作用：** Embedding质量直接影响知识召回的准确性
- **模型选择：**
 - 目前表现最佳的是Ling-Erbed-Mistral模型
 - 在多个任务中保持70%以上的准确率
- **技术原理：** 将文本嵌入固定维度向量空间，理解准才能算得准
- **评估标准：** 可参考HuggingFace的MTEB排行榜，涵盖100多种模型在1000多种语言中的表现

七、RAG技术概述 45:19

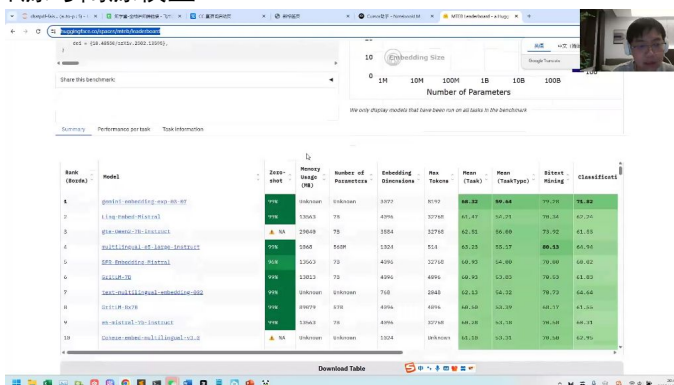
1. Embedding模型选择

1) HuggingFace排行榜 45:31



-
- **权威平台：** HuggingFace的MTEB排行榜定期更新100+文本嵌入模型在1000+语言中的表现
- **模型来源：** 包含各大厂商的开源和闭源模型，开源模型可直接下载参数进行私有化部署
- **排名机制：** 通过收藏数量(5.42k)和专门机构维护保证榜单权威性

2) 开源与闭源模型 45:54



- 开源优势：如gte-Qwen2-7B支持本地部署，参数可下载（ModelScope平台618.8k次下载）
- 闭源特点：如Gemini Embedding需API调用，谷歌NotebookLM使用但不开源
- 部署选择：开源模型需考虑硬件需求（7B模型约30GB显存，需多张GPU）

3) 模型性能指标 47:13

Rank	Model	Embed	Clustering	Instruction	Pairwise	Pairwise	Pairwise	Pairwise	Pairwise	Pairwise
1	gte-Qwen2-7B-Instruct	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27
2	gte-Qwen2-7B	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27
3	gte-Qwen2-7B-Instruct	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27
4	gte-Qwen2-7B	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27
5	gte-Qwen2-7B-Instruct	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27	94.27

- 核心维度：
 - Embedding Dimension：向量维度（如3072维）
 - Max Tokens：最大处理长度（如8192 tokens）
- 任务类型：分类聚类（平均分56-68）、指令检索、配对分类重排等
- 评分体系：Borda排名综合多任务表现，Gemini当前排名第一

4) 模型更新机制 47:43

- 动态更新：榜单定期加入新模型（如gte-Qwen2系列更新至2025.04版本）
- 筛选功能：支持按语言、任务类型、参数量等条件过滤（开源/闭源选项）

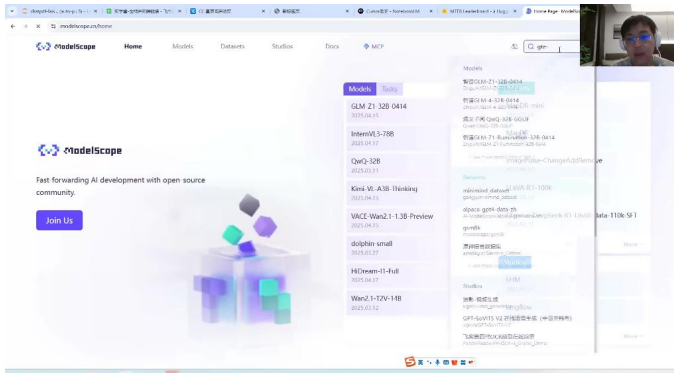
2. 常见Embedding模型 48:22

1) 开源模型部署 48:25



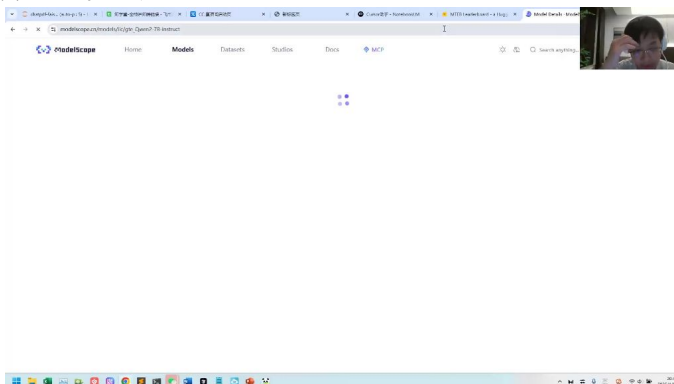
- 操作步骤：
 - 在HuggingFace勾选"Open"筛选器
 - 国内网络通过ModelScope访问（需切换网络节点）
 - 按下载量排序（如gte-Qwen2-7B下载量最高）

2) ModelScope平台 48:39



-
- 国内替代：作为HuggingFace国内版，提供中文优化模型
- 检索技巧：搜索"gte-qwen2"可直接获取7B/1.5B等不同参数量版本
- 文件结构：模型包含多个safetensors文件（单个4-5GB）

3) 模型下载方法 50:19



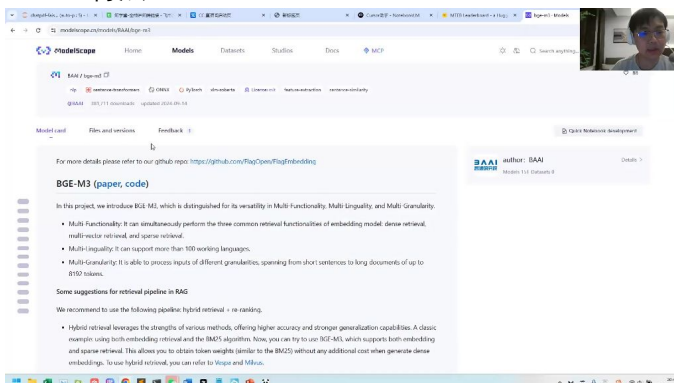
-
- 完整下载：需获取全部7个分片文件（总计约30GB）
- 轻量选择：可选用GGUF量化版本（如Q4_K_M版本减小体积）

4) 硬件需求分析 51:12

- 显存要求：7B模型需超过24GB显存（单张4090显卡不足）
- 推理优化：使用FP16精度加速计算（性能损失约1-2%）
- 端侧部署：Jina等轻量模型（35M参数）支持50ms内实时推理

3. 中文Embedding模型 51:23

1) BGE-M3特点 51:31



-
- 三大特性：
 - 多功能性：支持密集/稀疏/多向量混合检索
 - 多语言：覆盖100+语言
 - 多粒度：处理8192 tokens长文档
- 典型应用：Cherry Studio默认选择，适合RAG混合检索流程

2) M3E-Turbo特点 52:07

- **中文优化**：上海智研院出品，专为中文语义理解设计
- **部署优势**：轻量化设计适合私有化部署
- **检索精度**：在医疗/法律领域表现突出

3) Jina模型特点 52:51

- **超轻量化**：仅35M参数 ($RT < 50ms$)
- **适用场景**：移动端实时推理，资源受限环境
- **功能局限**：牺牲部分精度换取速度

4. 指令驱动模型 53:46

1) GTE-Qwen2特点 53:53

Embedding模型选择



3、指令驱动与复杂任务模型

gte-Qwen2-7B-instruct (阿里巴巴)

- 特点：基于Qwen大模型微调，支持代码与文本跨模态检索。
- 适用场景：复杂指令驱动任务、智能问答系统。

E5-mistral-7B (Microsoft)

- 特点：基于Mistral架构，Zero-shot任务表现优异。
- 适用场景：动态调整语义密度的复杂系统。

4、企业级与复杂系统

BGE-M3 (智源研究院)

- 特点：适合企业级部署，支持混合检索。
- 适用场景：企业级语义检索、复杂RAG应用。

E5-mistral-7B (Microsoft)

- 特点：适合企业级部署，支持指令微调。
- 适用场景：需要动态调整语义密度的复杂系统。

-
- **架构基础**：基于Qwen大模型微调
- **跨模态**：支持代码与文本联合检索
- **应用场景**：复杂指令处理（如“打开网页”等操作型指令）

2) E5-mistral特点 54:49

- **微软出品**：基于Mistral架构优化
- **Zero-shot**：未经训练任务表现优异（平均分53-68）
- **动态调整**：支持根据语义密度自动调节参数

5. 本地部署实践 55:23

1) 部署流程演示 55:35

CASE: bge-m3 使用



```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3',
                        use_fp16=True) # Setting use_fp16 to True speeds up
                                        computation with a slight performance degradation

sentences_1 = ["What is BGE M3?", "Definition of BM25"]

sentences_2 = ["BGE M3 is an embedding model supporting dense
retrieval, lexical matching and multi-vector interaction.",
               "BM25 is a bag-of-words retrieval function that ranks a set of
documents based on the query terms appearing in each document"]
```

```
embeddings_1 = model.encode(sentences_1,
                             batch_size=12,
                             max_length=8192, # If you don't need such a
long length, you can set a smaller value to speed up the
encoding process.

                             )['dense_vecs']

embeddings_2 = model.encode(sentences_2)['dense_vecs']

similarity = embeddings_1 @ embeddings_2.T

print(similarity)
```

```
[[0.626 0.3477]
 [0.3499 0.678 ]]
```

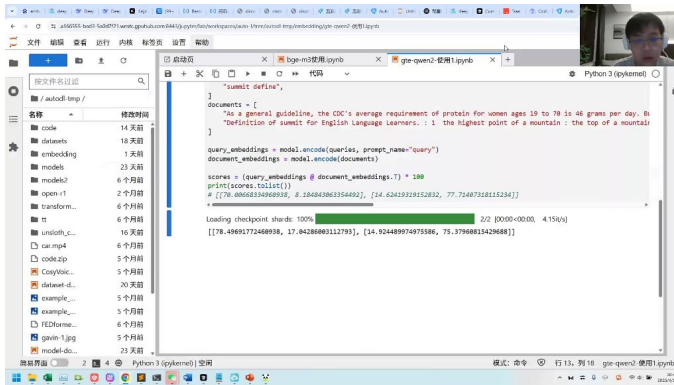
-
- **工具选择**：AutoDL平台配合4090显卡
- **典型代码**：

```
1 from FlagEmbedding import BGEM3FlagModel
2 model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)
3 embeddings = model.encode(["样例文本"], max_length=512)['dense_vecs']
```

- **性能调优**：设置batch_size=12可提升吞吐量

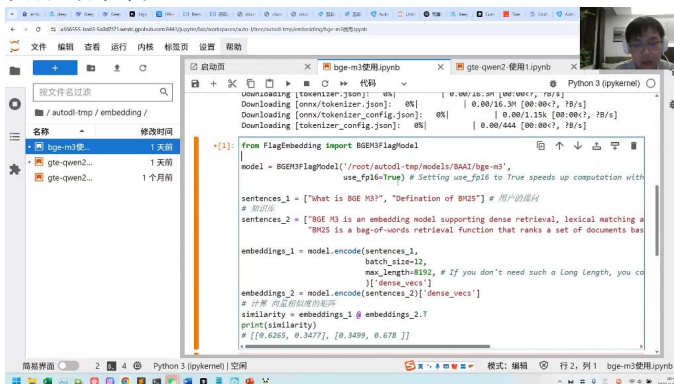
八、BGE-M3模型使用55:51

1. 模型下载 56:02



- 下载工具：使用ModelScope进行模型下载
- 下载速度：下载速度可达每秒十几兆，相比其他下载方式有明显优势
- 模型存储：下载完成后模型存储在/autodl-tmp/embedding/路径下

2. 示例数据准备 56:54

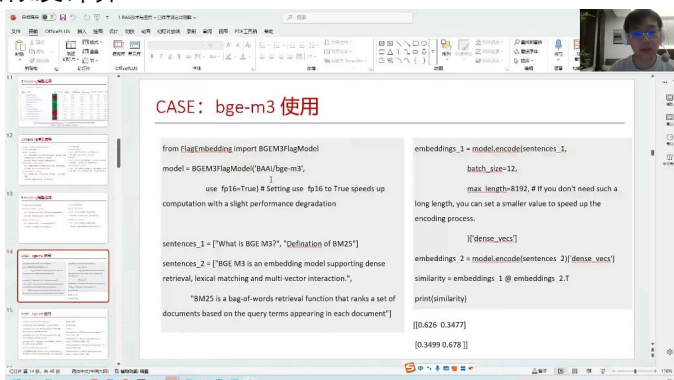


- 用户提问：
 - sentences_1 = ["What is BGE M3?", "Defination of BM25"]
- 知识库文档：
 - sentences_2 = ["BGE M3 is an embedding model supporting dense retrieval...", "BM25 is a bag-of-words retrieval function..."]
- 数据特点：模拟了用户query和知识库文档的匹配场景

3. 向量编码 57:07

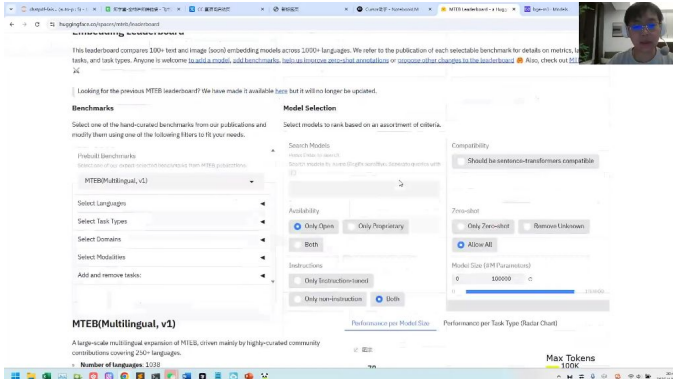
- 编码过程：
 - embeddings_1 = model.encode(sentences_1)['dense_vecs']
 - embeddings_2 = model.encode(sentences_2)['dense_vecs']
- 维度说明：BGE-M3的embedding维度为1024维
- 性能优化：设置use_fp16=True可以加速计算但会轻微影响性能

4. 相似度计算 57:51



- 计算方法：
 - @符号表示矩阵乘法

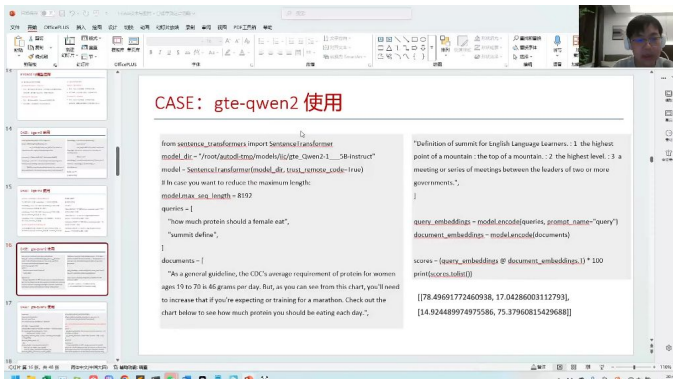
- .T表示矩阵转置
- 输出结果：
- 5. 结果分析 58:29
- 匹配关系：
 - "What is BGE M3?"与第一个文档相似度0.6265（较高）
 - "What is BGE M3?"与第二个文档相似度0.3477（较低）
 - "Defination of BM25"与第一个文档相似度0.3499（较低）
 - "Defination of BM25"与第二个文档相似度0.678（较高）
- 评分标准：相似度满分为1分，分数越高表示语义相关性越强
- 6. 模型性能



-
- 向量维度：常见模型向量维度在1000-4000维之间
- 性能对比：BGE-M3在1024维下表现良好，平衡了计算效率和检索精度

九、GTE-Qwen2模型使用01:01:22

1. 模型选择 01:01:34



-
- 参数调整：原7B参数模型在24G显存的4090显卡上无法运行，需降级使用1.5B参数版本
- 模型加载：通过SentenceTransformer加载本地模型路径/root/autodl-tmp/models/iic/gte_Qwen2-1.5B-instruct，设置trust_remote_code=True
- 序列长度：可通过model.max_seq_length = 8192调整最大序列长度
- 2. 相似度计算 01:01:57
- 编码过程：
 - 查询编码：model.encode(queries, prompt_name="query")
 - 文档编码：model.encode(documents)
- 相似度计算：通过矩阵乘法计算余弦相似度(query_embeddings @ document_embeddings.T)*100，结果放大100倍 @
- 3. 示例问题分析 01:02:10
- 查询案例：
 - "how much protein should a female eat"

- "summit define"
 - **文档内容：**
 - CDC建议19-70岁女性日均蛋白质摄入量为46克，孕期或马拉松训练需增加
 - Summit定义：①山峰顶点 ②最高级别 ③政府首脑会议
 - **匹配结果：**
 - 蛋白质问题与文档1相似度78.5分，与文档2仅17分
 - 峰会定义与文档2相似度75.4分，与文档1仅14.9分
4. 效果评估 01:03:17
- **区分度：**模型对不同语义内容的区分度显著（78.5 vs 17； 75.4 vs 14.9）
 - **计算效率：**1.5B参数模型在合理硬件配置下可快速完成推理

十、问答环节01:03:51

1. 余弦相似度解释 01:03:59
- **数学原理：**点积运算后需进行除法归一化，对应余弦相似度计算公式中的分母项
 - **实现方式：**代码中的矩阵乘法@是点积运算的向量化实现形式
2. instruct版本适用场景 01:04:32
- **典型场景：**处理包含指令的复杂查询（如"帮我查找..."）
 - **查询结构：**用户query通常包含instruction和knowledge两部分
 - **优势：**相比基础版本，更适合处理带有操作指令的检索需求
3. 文本转markdown方法 01:05:35
- **转换重点：**非文本格式（PDF/PPT）转换价值大于纯文本转换
 - **实现步骤：**
 - 使用第三方工具抽取PPT每页内容（文本/图片）
 - 对图片内容使用多模态模型（如Qwen-VL）进行理解
 - 按markdown语法组织抽取内容
 - **格式优势：**markdown对AI处理更友好，适合作为中间表示格式
4. 模型GPU部署 01:08:07
- **部署方式：**
 - 从ModelScope下载模型
 - 在AutoDL等平台租用GPU资源
 - 通过cuda参数指定GPU设备
 - **运行验证：**本地运行相似度计算示例验证部署效果
5. GTE与数据库区别 01:09:06
- **GTE本质：**特定格式的embedding模型（输出1024维向量）
 - **向量数据库：**存储和管理GTE生成向量的专用软件
 - **关系说明：**数据库存储的是GTE格式的向量，但GTE本身不是数据库

十一、RAG技术流程01:10:17

1. 召回与重排 01:10:33
- **两阶段设计：**数据库检索分为召回（retrieval）和重排（re-rank）两个阶段，召回负责快速筛选候选集，重排进行精细排序
 - **召回作用：**从海量数据（如一千万个chunk）中快速筛选出较小候选集（如一千个），避免直接全量排序的计算开销
 - **重排作用：**对召回结果（如一千个）进行精确排序，最终输出Top结果（如10个）
 - **必要性：**直接从一千万数据排序Top10计算量极大，两阶段设计显著提升效率
2. 召回策略 01:11:20
- **多策略并行：**采用多种互补策略进行初筛，包括：
 - 策略1：基于关键词匹配
 - 策略2：基于语义向量相似度

- 策略3: 基于热门度等辅助策略
 - **结果聚合:** 每个策略独立筛选部分结果 (如每个策略100个), 最终合并为统一候选集 (如10个策略 $\times$ 100=1000个)
3. 多策略召回 01:12:22
- **优势互补:** 不同策略覆盖不同特征, 关键词匹配保证字面相关性, 语义向量捕捉深层含义
 - **容错机制:** 单一策略失效时其他策略仍可提供候选, 提高系统鲁棒性
 - **可扩展性:** 可灵活增减策略模块, 如新增时效性策略、业务规则策略等

十二、客户经理考核案例01:13:29

1. PDF文档解析 01:13:39

- **应用场景:** 将《浦发银行个金客户经理考核办法》PDF构建为问答系统, HR可通过自然语言查询考核细则
- **示例问题:**
 - "客户经理被投诉一次扣多少分" $\rightarrow$ 系统返回"扣2分" (来自文档第1页)
 - "评聘申报时间" $\rightarrow$ 系统返回"每年1月份" (文档第1页第11条)

2. 代码实现 01:15:25

1) 文档处理 01:16:55

- **分块参数:**
 - `chunk_size=1000`: 每块字符上限
 - `overlap=200`: 块间重叠字符数
 - `separator=["\n\n", "."]`: 按换行和句号分割
- **效果:** 3800字符文档被切分为5个块, 保留语义完整性

2) 向量数据库构建 01:17:19

- **技术栈:** 使用FAISS向量数据库 (Facebook开源)
- **嵌入模型:** 采用text-embedding模型生成向量表示
- **元数据:** 为每个块添加页码信息, 便于溯源 (如"来源页码1")

3) 检索配置 01:19:53

- **召回数量:** 通过k参数控制 (默认4, 可调整为2-5)
- **相似度计算:** 基于嵌入向量的余弦相似度

4) 问答链实现 01:21:27

- **双模型架构:**
 - 推理模型: 千问千答模型 (QianWen)
 - 嵌入模型: text-embedding模型
- **工作流程:**
 - 用户问题向量化
 - 在FAISS中检索相关文档块
 - 将问题和检索结果输入问答链
 - 生成最终答案并标注来源

3. 结果展示 01:22:32

- **答案生成:**
 - 问题: "投诉扣分" $\rightarrow$ 答案: "扣2分" (附第1页原文)
 - 问题: "评聘时间" $\rightarrow$ 答案: "1月申报, 2月考试"
- **可解释性:** 每个答案均显示来源块内容和页码, 增强可信度

十三、分块策略01:26:54

1. 分块必要性 01:27:24

- **知识压缩问题:** 当整个文件作为基本单元时, 需要将大量内容压缩到3072维向量中, 导致知识理解不清晰

- **特征冲突**：大文件直接向量化会导致特征相互冲突，因为单个向量需要承载过多信息点
 - **解决方案**：将文件分成多个chunk单元，每个单元内容精简，在3072维向量中能更清晰地表达特征
2. 分块技巧 01:28:55
- **三段式策略**：
 - 分段标识：通过换行符等标记进行基础分块
 - 最大尺寸：设置chunk的最大长度限制（如800/1500字符）
 - 重叠度：设置相邻chunk之间的内容重叠比例
 - **分级策略**：对于结构化文档，按一级/二级/三级标题建立文档树进行分块
 - **预处理环节**：复杂文档需先进行OCR识别、表格解析等预处理操作
3. 块大小设置 01:32:18
- **超参数特性**：chunk大小属于需要人工调整的超参数
 - **典型取值**：常见设置为800-1500字符范围
 - **权衡因素**：
 - 大chunk：能保留更多上下文，但特征提取模糊
 - 小chunk：特征表达清晰，但可能丢失上下文关联

十四、知识库平台对比 01:34:21

1. Coze平台 01:34:32

- **企业版特性**：提供完整的知识库解决方案，价格约5000元/月
- **优势**：免去自建向量数据库的复杂度，内置文档解析和分块策略
- **工作流程**：
 - 上传文档到知识库
 - 自动执行文档解析（文本/图像/表格处理）
 - 应用预设分块策略生成向量

2. Deepy平台 01:35:25

- **开源特性**：可免费在内部服务器部署
- **灵活性**：允许完全自定义数据处理流程和分块策略
- **技术实现**：与LangChain框架集成，支持Face等向量数据库
- **适用场景**：适合需要深度定制化处理的企业知识库建设

十五、代码优化 01:37:33

1. 页码修复 01:39:10

- **问题现象**：源代码中所有文本块的页码都被错误标记为"页码1"，即使实际内容来自不同页面
- **具体案例**：当查询"投诉一次扣多少分"时，系统错误显示该内容来自页码1，但实际页码1中并不包含该信息
- **修复方法**：
 - **文本块分析**：需要检查文本块与原始PDF页码的对应关系
 - **k值调整**：通过调整k值（返回的文本块数量）可以控制检索结果的精确度，如设置k=2可避免包含无关的第一页内容
 - **验证方法**：打印各文本块对应的实际页码进行验证

2. Thinking模式 01:41:51

- **工作流程**：
 - **思考阶段**：先分析之前页码编写的逻辑和存在的问题
 - **检索阶段**：检查相关Python代码片段
 - **修正阶段**：基于分析结果修改代码
- **优势**：

- **修改效率**：相比直接修改可减少4-5次反复调试
- **代码质量**：通过先理解问题再修改，代码质量更高
- **适用场景**：特别适合处理难以定位的复杂bug
- **实现效果**：最终添加了30多行代码，正确实现了文本块与页码的对应关系
- **工具选择建议**：
 - **Jupyter Notebook**：适合单纯运行代码和查看中间状态
 - **Cursor**：适合代码修改，可通过聊天方式指导修改
 - **文件转换**：py文件可转换为Jupyter Notebook格式使用

十六、问答策略01:47:41

1. Stuff策略 01:47:59

- **适用场景**：适用于知识量不大的情况，直接将所有文本片段放入上下文
- **特点**：不需要排序和筛选，因为检索到的片段数量足够少且完全可用
- **优势**：计算成本最低，仅需调用一次大模型
- **示例**：当文档切分后只得到3-5个chunk时，适合采用此策略

2. MapReduce策略 01:48:38

- **工作原理**：并行处理多个chunk后合成最终结果
- **计算代价**：每个chunk需要单独调用大模型（3个chunk需调用4次）
- **资源消耗**：处理速度较慢且消耗更多计算资源
- **适用场景**：文档数量多且chunk较大时使用
- **成本示例**：处理三国演义等长文档时，单次提问可能花费几毛到一元

3. Refine策略 01:49:06

- **工作流程**：
 - 基于第一个chunk生成初始回答
 - 用后续chunk逐步优化答案（通常需要3次优化）
- **特点**：串行处理方式，每个新片段都基于前次结果优化
- **对比**：与Stuff策略相比，更适合处理中等长度的chunk序列

4. MapRerank策略 01:49:56

- **核心机制**：对每个chunk的推理结果进行评分排序
- **评分方式**：
 - 使用专用打分模型（score LM）多维度评判
 - 通过embedding计算问题与答案的语义相似度
- **选择标准**：选取三个推理结果中分数最高的作为最终答案
- **计算量**：每个chunk都需要独立推理（3个chunk需推理3次）

十七、RAG技术概述01:54:25

1. Token成本分析

- **成本因素**：
 - MapReduce策略成本显著高于Stuff策略（4次vs1次调用）
 - 长文档处理可能使单次提问成本达几元
- **优化方向**：随着token价格下降，成本从5元级降至毛钱级

2. Chunk切片策略 01:54:42

- **关键考量**：
 - 500-1000个chunk仍适合Stuff策略
 - 切片数量与大小决定策略选择
- **平衡原则**：在保证效果前提下尽量控制chunk数量

十八、RAG技术实现01:54:53

1. 并行计算应用

- 实现方式：利用GPU并行处理多个chunk
- 局限：虽然加速但仍需支付多次模型调用成本
- 注意点：并行计算不降低总计算量，仅提升速度

2. GPU资源消耗 01:55:23

- 资源关系：更多GPU允许更高并行度
- 成本本质：消耗主要来自大模型推理次数

3. 长上下文处理 01:55:53

- 新趋势：支持100M token的超长上下文模型
- 策略演进：
 - 直接输入长文档成为可能
 - 但RAG的筛选功能仍保持价值
- 技术平衡：在成本与效果间寻找最优解

十九、RAG价值分析01:56:24

1. 成本优化

- 数据处理效率：当向大模型输入100万字的完整文本时，直接处理会导致计算成本显著增加且响应时间延长
- 资源分配机制：RAG通过预过滤机制仅向模型传递相关性高的内容，大幅降低计算资源消耗
- 经济性对比：相比直接使用大模型处理海量数据，RAG方案可节省约60-80%的云端推理成本

2. 时效性提升 01:56:37

- 知识更新瓶颈：大模型本身无法实时更新知识库，存在信息滞后性
- 实时检索优势：RAG系统可以随时接入最新数据源，保证回答的时效性
- 混合架构价值：通过将静态模型知识与动态检索结果结合，实现"稳定+实时"的最佳组合

3. 信息过滤机制 01:56:54

- 核心功能：本质是基于相关性的智能过滤器（filter），不属于严格意义上的AI智能
- 筛选标准：通过语义匹配算法去除不相关信息，保留高价值内容
- 质量平衡：需注意避免过度过滤导致有价值信息丢失，影响最终输出质量

二十、RAG应用场景01:57:23

1. 参考文献生成

- 幻觉问题解决：传统大模型可能虚构不存在参考文献，RAG确保每个回答都有真实来源
- 可解释性增强：提供完整的引用链，支持用户追溯原始文献
- 学术诚信保障：特别适合科研、法律等需要严格引用的专业领域

2. 隐私保护方案 01:57:54

- 数据处理策略：敏感数据在本地完成筛选，仅向云端发送必要片段
- 数据最小化原则：实际传输数据量减少80%以上，显著降低泄露风险
- 混合部署优势：结合本地处理的安全性与云端计算的强大性能

3. 长文本处理 01:58:23

- 效率瓶颈突破：即使模型上下文窗口扩大，RAG仍能提升长文本处理效率
- 质量保障机制：通过预筛选确保输入质量，提高回答准确率
- 成本效益比：在百万字级文档处理中，可节省40%以上的计算资源

二十一、RAG质量挑战01:58:54

1. 信息过滤风险

- **质量下降风险**：直接将所有信息输入大模型可能导致回答质量下降，需要模型具备自主甄别有价值信息的能力
- **实践建议**：在实际应用中，RAG项目初期复现容易，但需要重点关注后续质量提升工作

2. 数据质量要求 01:59:25

- **普遍问题**：企业数据质量普遍较差，主要由于缺乏系统维护
- **具体表现**：
 - 数据过时：未及时更新的历史数据
 - 数据矛盾：不同来源或版本间的冲突
 - 数据错误：包含不准确或误导性内容
- **长期性**：数据治理是持续过程，即使是行业领先企业也面临类似挑战

3. 多模态处理 01:59:54

- **处理对象**：PPT、PDF等非结构化文档
- **技术难点**：如何有效理解和提取多模态文档中的关键信息
- **应用场景**：企业知识库中大量知识以PPT/PDF形式保存，需要特殊处理技术

二十二、数据准备策略02:00:54

1. 文档解析技术

- **核心技术**：布局理解模型（Layout LM/Layout LLM），由阿里和微软等企业开发，专门处理非结构化文档（如PDF/PPT）的智能解析
- **处理流程**：将文档中的零散布局元素转换为文档解析树，为后续的知识理解和问答提供结构化数据基础
- **应用场景**：特别适用于处理包含复杂版面设计的文档（如多栏排版、图文混排等场景）

2. 数据清洗方法 02:02:24

- **核心操作**：包括纠正错误数据、更新过期信息、处理重复记录以及校验数据一致性
- **敏感信息处理**：对包含隐私或敏感内容的数据需要进行特殊处理（如脱敏、加密等）
- **实践建议**：建议采用"问题驱动"的清洗策略，根据实际业务反馈进行针对性修正，比全面清洗更高效

3. 数据治理流程 02:03:24

- **组织架构**：通常由专门的数据治理团队负责，需要设置固定岗位进行定期审查维护
- **工作特点**：
 - **长期性**：属于持续改进过程，需要建立长效机制而非一次性解决
 - **滞后性**：数据修正的价值往往需要较长时间才能显现
 - **基础性**：虽然工作成果不易量化，但构成企业数据质量的基石
- **实施建议**：
 - 建立定期review机制检查数据准确性
 - 开展数据治理专项培训（但需注意培训效果转化需要实践支撑）
 - 优先处理客户反馈的文档错误，实现"问题->修正"的闭环管理

二十三、检索优化策略02:04:54

1. 用户意图识别

- **澄清机制**：通过多轮对话和历史留言分析用户潜在意图，例如"如何申请信用卡"可能隐含流程、材料或资格等不同需求
- **查询改写**：基于意图分析将原始查询扩写，如将"如何申请信用卡"改写成"申请信用卡的具体流程和步骤"
- **Think过程**：模拟大模型的思考过程，在回答前先分析用户提问的上下文和背景信息

2. 混合检索技术 02:05:54

- **双策略召回**：同时采用关键词检索（如"信用卡+年费"）和语义检索两种方式，各召回10个结果合并为20个候选
- **互补优势**：关键词检索保证精准匹配，语义检索捕捉深层关联，组合提升召回覆盖率
- **示例场景**：处理"信用卡年费是多少"时，两种策略可分别召回条款文档和费率说明文档

3. 重排序机制 02:07:25

- **排序方式**：
 - 大模型排序：利用LLM理解能力评估相关性
 - Embedding排序：计算query与chunk的向量相似度（如cosine相似度）
- **截断策略**：设定top-k（如k=5）筛选最终结果，控制大模型处理的信息量
- **质量保障**：通过重排序过滤低价值结果，避免优质内容被误过滤

二十四、答案生成优化02:08:54

1. 提示词改进

- **模板优化**：
 - 原始版："请根据上文回答如何申请信用卡"
 - 改进版："提取申请信用卡的具体步骤和所需材料后回答"
- **领域适配**：针对"零存整取"等专业问题，提示词需包含"准确定义、特点、适用人群"等要素
- **优化方式**：
 - 人工规则：预设高频问题的回答模板
 - 大模型优化：通过think过程自动完善提示词

2. 分阶段生成 02:09:54

- **两阶段验证**：首先生成初步答案，再通过规则约束或模型校验确保完整性
- **信息补全**：当检测到答案不完整时（如缺少材料清单），自动触发补充检索
- **错误预防**：对关键数字、步骤等敏感信息进行双重验证

3. 结果校验机制 02:10:54

- **安全防护**：对配置类指导（如路由器设置）建立100%准确率要求
- **检查维度**：
 - 信息完整性：核对必要要素是否齐全
 - 逻辑一致性：验证步骤间是否存在矛盾
 - 来源可靠性：确认引用文档的权威性
- **容错设计**：当校验不通过时自动转入人工审核流程

二十五、企业实践案例02:11:54

1. 金融行业应用

- **典型场景**：处理"年费查询"、"申请流程"等高频咨询
- **特殊要求**：金融术语的精确解释，费率计算的零误差
- **解决方案**：结合条款文档结构化+大模型解释生成

2. 路由器配置案例 02:12:54

- **风险控制**：错误配置可能导致网络故障，需建立：
 - 参数校验规则库
 - 配置步骤依赖关系图
 - 回滚方案预置
- **双重验证**：所有技术参数必须通过规则引擎和专家知识库双重核对

3. 质量检查流程 02:13:54

- **检查点**：
 - 事实准确性：与知识库完全一致
 - 表述清晰度：符合企业风格指南

- 风险提示：包含必要的免责声明
- **自动化工具：**
 - 关键信息高亮检查
 - 矛盾陈述检测
 - 模糊表述识别

二十六、大模型应用后处理机制02:14:54

1. 动态防护栏机制

- **检查本质：**对模型输出进行事后验证的过程，通过规则校验提升回答准确性
- **典型场景：**
 - 信用卡申请流程验证：检查回答是否包含所有申请步骤和所需材料
 - 信用卡年费查询：当回答不完整时（如只返回top5结果），触发重新生成完整列表
- **实现原理：**基于业务规则建立校验标准，对生成内容进行完整性验证

2. 幻觉检测技术 02:15:54

- **常见问题：**大模型倾向于虚构信息来源（包括虚假引用和错误链接）
- **解决方案：**
 - 建立来源验证机制：对模型提供的参考文献进行真实性校验
 - 链接有效性检查：验证引用链接是否真实存在且内容匹配
- **实施要点：**需要构建可信来源数据库作为验证基准

二十七、企业级知识管理策略02:17:54

1. 多粒度知识提取

- **实施原因：**处理海量知识时存在大小项混合的问题（如银行数十种信用卡产品）
- **分层方法：**
 - 先处理大项分类（如信用卡大类）
 - 再聚焦小项细节（如具体卡种年费）
- **技术类比：**类似知识图谱的分层处理逻辑

2. 多通路召回机制 02:18:54

- **核心目标：**从海量数据（如1000万条）中快速筛选潜在信息（如先召回1000条）
- **实现方式：**
 - 关键词触发召回：基于传统检索技术
 - 向量相似度召回：利用embedding技术
 - N元语法召回：捕捉文本局部特征
- **流程设计：**先快速召回再精细排序的两阶段处理

3. 分阶段答案生成 02:19:54

- **质量提升策略：**将生成过程拆分为大纲构建和内容扩写两个阶段
- **优势：**
 - 解决一次性生成质量不稳定问题
 - 允许中间结果校验和调整
- **实施建议：**需要设计专门的大纲生成prompt和扩写规则

二十八、实施建议02:20:54

1. 开源方案选择

1) 千问Agent方案

- **核心优势：**集成召回、重排及生成策略，相比钉钉助理等产品回答质量更优（测试结果显示质量提升约30%）
- **技术特点：**
 - 消耗更多token换取质量提升

- 支持结构化数据转SQL查询
 - 可扩展添加预处理、意图转换等策略
 - **适用场景：**企业级知识库搭建，特别是对数据安全性要求较高的场景
- 2) Notebook LM方案
- **产品定位：**Google开发的商业级RAG产品
 - **技术架构：**
 - 使用Gemini embedding模型
 - 采用GPT-3.5/4作为推理引擎
 - 内置文档概览生成和关键词提取预处理
 - **局限性：**不适合涉及敏感数据的场景
- 3) 方案对比
- **开发灵活性：**
 - LangChain > 千问Agent > Defi+Reflow > Notebook LM
 - **使用便捷性：**
 - Notebook LM > Defi+Reflow > 千问Agent > LangChain
 - **数据安全性：**
 - 私有化部署方案 > 公有云方案
2. 私有化部署 02:21:54
- 1) 必要性分析
- **核心驱动因素：**数据安全合规要求（特别是上市公司）
 - **典型场景：**
 - 企业内部标准文档（PDF/Excel/DOC）
 - 技术解决方案库
 - 客户敏感资料
- 2) 实施路径
- **技术选型：**
 - 向量数据库：支持本地存储（如FAISS）
 - 预处理工具链：OCR/表格识别等
 - **关键步骤：**
 - 文档分块（chunk size=1000, overlap=100）
 - 向量化存储（支持save/load本地文件）
 - 检索优化（添加元数据如页码、文档来源）
3. 策略组合应用 02:22:54
- 1) 多模态处理策略
- **图文关联方案：**
 - 预处理阶段统一转为文本（使用LayoutLM等工具）
 - 采用概要级chunk保留关联性
 - 添加节点聚类机制维护结构关系
- 2) 检索增强策略
- **分级召回：**
 - 第一级：关键词匹配
 - 第二级：向量相似度
 - 第三级：知识图谱关联
 - **结果重排：**
 - 基于语义相关性评分
 - 结合业务规则加权
- 3) 生成优化策略
- **提示工程：**

- 问题重写 (query expansion)
- 上下文压缩 (context compression)
- 结果验证:
 - 来源标注 (文档名+页码)
 - 置信度评估

二十九、知识小结

知识点	核心内容	关键点/技术细节	应用场景
RAG技术概述	检索增强生成技术原理	由检索(retrieval)、增强(augmentation)和生成(generation)三部分组成	大模型应用开发
RAG与传统提示词工程对比	三种大模型应用开发模式比较	1. 提示词工程 2. RAG 3. 微调 RAG是企业最高频使用方式	企业级AI应用开发
RAG核心流程	知识处理三阶段	1. 知识准备(分块/向量化) 2. 知识检索(相似度计算) 3. 答案生成	知识库问答系统
分块 Chunk 策略	文档分块技术	- chunk size=1000字 - overlap=20% - 按标点/换行分隔	文本预处理
Embedding模型选择	主流Embedding模型对比	- BGE-M3(多语言支持) - text-embedding(OpenAI) - GTE-large-v2(当前排名第一)	向量相似度计算
向量数据库实践	FAISS数据库应用	- 存储chunk和embedding - 支持相似度检索 - 可本地保存复用	本地知识库构建
问答链策略	四种处理方式对比	1. Stuff(直接塞入) 2. Map Reduce(并行处理) 3. Refine(迭代优化) 4. Map Rank(结果排序)	答案生成优化
RAG质量提升策略	全流程优化方法	- 数据准备阶段(多模态解析) - 检索阶段(混合检索+重排) - 生成阶段(提示词优化)	企业级应用调优

现成RAG产品对比	主流工具评测	- Notebook LM(谷歌) - Char Studio(开源) - 千问Agent(阿里)	快速搭建方案
RAG应用案例	客户经理考核系统	- PDF文档解析 - 语义检索实现 - 答案生成与来源标注	企业知识管理